

EC ENG 232E - Summer 25 - Project 4

August 29, 2025

Team members:

Kaylee Lo (UID: _____)

Joanna Tang (UID: _____)

Rachel Zhang (UID: _____)

Part I

Stock Market

```
[1]: if (!require("igraph")) install.packages("igraph")
library ("igraph")
if (!require("clevr")) install.packages("clevr")
library ("clevr")
# https://cran.r-project.org/web/packages/clevr/index.html

[3]: path = "finance_data/data/"
out.file<-" "
file.names <- dir(path, pattern = ".csv")
m <- matrix(, nrow = 0, ncol = 765)
sectors.table <- read.table("finance_data/Name_sector.csv",header=TRUE, sep="," ,
↪stringsAsFactors=TRUE)
sectors = c()
sectors.names <- c()
for(i in 1:length(file.names)){
  file <- read.table(paste("finance_data/data/",file.names[i],sep =
↪""),header=TRUE, sep="," , stringsAsFactors=FALSE)
  if(length(file$Close)==765){
    m <- rbind(m, matrix(file$Close, nrow=1, ncol=765))
    mystr <- substr(file.names[i], 1, nchar(file.names[i])-4)
    sector <- sectors.table$Sector[which(sectors.table$Symbol == mystr)]
    # print(sector)
    # sectors = c(sectors, as.factor(sector))
    sectors.names = c(sectors.names, toString(sector))
  }
}
```

```

}
com_num = length(sectors.names)
data_num = ncol(m)
sector.set<-as.factor(sectors.names)
sectors.index<-as.numeric(sector.set)
num_sector <- length(unique(sectors.index))
table(sector.set)

# 2014/5/1 - 2017/5/12
# 3 min

```

```

sector.set
  Consumer Discretionary      Consumer Staples
                84                36
                Energy          Financials
                34                63
                Health Care      Industrials
                60                64
  Information Technology      Materials
                66                24
  Real Estate Telecommunication Services
                31                4
                Utilities
                28

```

```

[4]: #unique(sectors.table) # 505x2 # symbol, sector
# length(sectors.names) # 494
# length(sector.set) # 84+34+60+66+31+28+36+63+64+24+4 = 494
# m #494 x 765
# num_sector #11
com_num

```

494

1 Return Correlation

1.1 Q1. What are upper and lower bounds on ρ_{ij} ?

Answer: ρ_{ij} has an upper bound of 1 and a lower bound of -1. Since we are comparing across multiple sectors, it is more consistent to use the log-normalized return rather than the regular return when comparing stocks.

Provide a justification for using log-normalized return ($r_i(t)$) instead of regular return ($q_i(t)$).

$$p_i(t)$$

is the closing price of stock i at the i^{th} day.

$$q_i(t)$$

is the return of stock i over a period of $[t-1, t]$.

$$q_i(t) = \frac{p_i(t) - p_i(t-1)}{p_i(t-1)}$$

$$r_i(t)$$

is the log-normalized return stock i over a period of $[t-1, t]$.

$$r_i(t) = \log(1 + q_i(t))$$

correlation

$$\rho_{ij} = \frac{\langle r_i(t)r_j(t) \rangle - \langle r_i(t) \rangle \langle r_j(t) \rangle}{\sqrt{(\langle r_i(t)^2 \rangle - \langle r_i(t) \rangle^2)(\langle r_j(t)^2 \rangle - \langle r_j(t) \rangle^2)}}$$

```
[5]: #correlation
cal_cor<-function(comp_date){
  #write code
  p <- comp_date # get closing price from comp_date, should have n stocks x m
  #dates
  m <- ncol(p)
  n <- nrow(p)
  print(paste("cal cor n x m:", n, m))
  q <- (p[, 2:m] - p[, 1:(m-1)]) / p[, 1:(m-1)] # list of return of stock
  #over a period of [t-1, t] matrix size should be n stocks x (m-1) dates
  r <- log(1+q) # list of log-normalized return stock over a period of [t-1,
  #t] matrix size should be n stocks x (m-1)
  #print(p[1,])
  #print(q[1,])
  #print(r[1,])

  rho = matrix(, nrow = n, ncol = n) # correlation matrix, size should be n x
  #n
  # since it is symmetric, use lower triangle only in loop
  for (i in c(1:n)){
    for (j in c(1:i)){ # use i not n
      rho[i,j] = (mean(r[i,] * r[j,]) - mean(r[i,]) * mean(r[j,])) /
      sqrt((mean(r[i,]^2) - mean(r[i,])^2) *
      (mean(r[j,]^2) - mean(r[j,])^2))
      if (i != j)
        rho[j,i] = rho[i,j]
    }
  }
  return(rho)
}
```

```
[6]: #m #494x765
#sectors.table #505
correlation_matrix <- cal_cor(m)
```

```
dim(correlation_matrix)
```

```
[1] "cal cor n x m: 494 765"
```

```
1. 494 2. 494
```

2 Constructing correlation graphs

The correlation graph has the stocks as the nodes and the edge weights are given by the following expression:

$$w_{ij} = \sqrt{2(1 - \rho_{ij})}$$

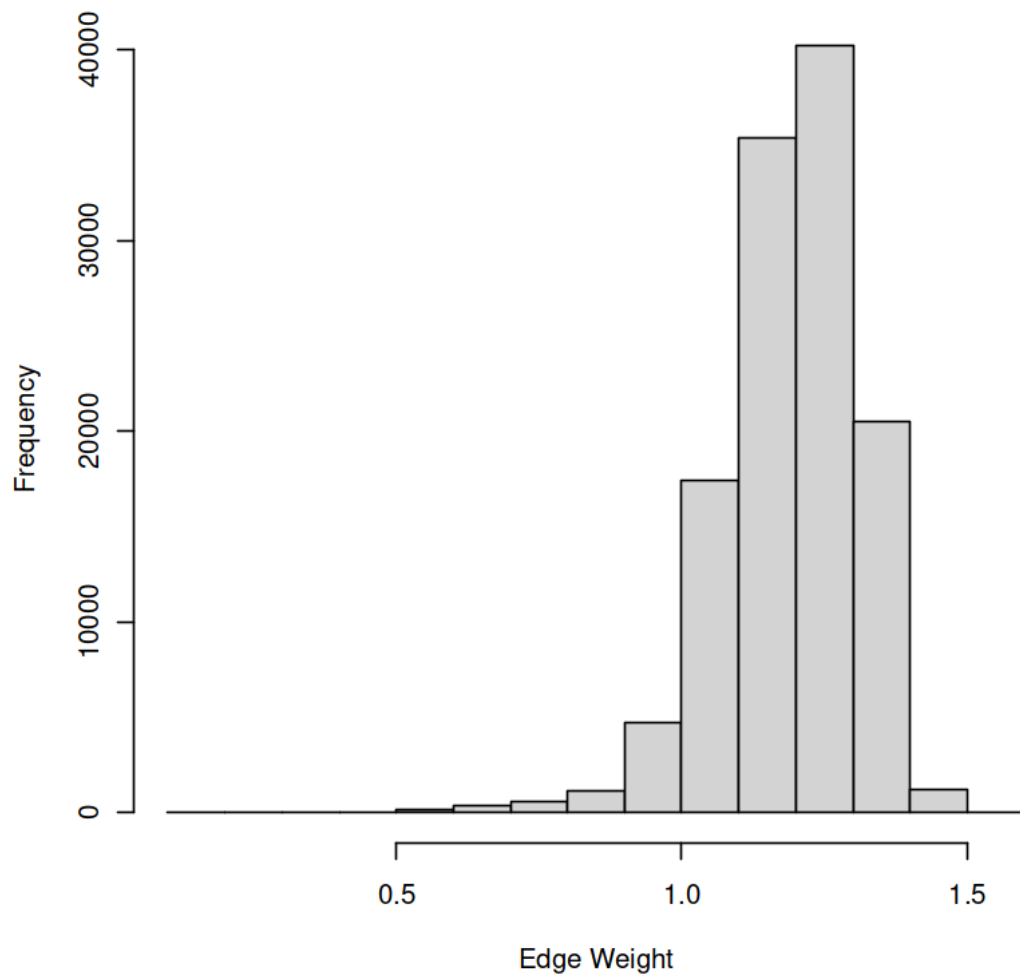
Compute the edge weights using the above expression and construct the correlation graph.

2.1 Q2. Plot a histogram showing the un-normalized distribution of edge weights.

```
[7]: w <- matrix(, nrow = com_num, ncol = com_num)
for (i in c(1:com_num)){
  for (j in c(1:i)){
    w[i,j] = sqrt(2*(1 - correlation_matrix[i,j]))
    w[j,i] = w[i,j]
  }
}
#w
g <- graph_from_adjacency_matrix(w, mode = "upper", weighted = TRUE, diag = TRUE,
  add.colnames = NULL, add.rownames = NA)
hist(E(g)$weight, main = "Unnormalized Distribution of Edge Weights (Daily)",
  xlab = "Edge Weight")

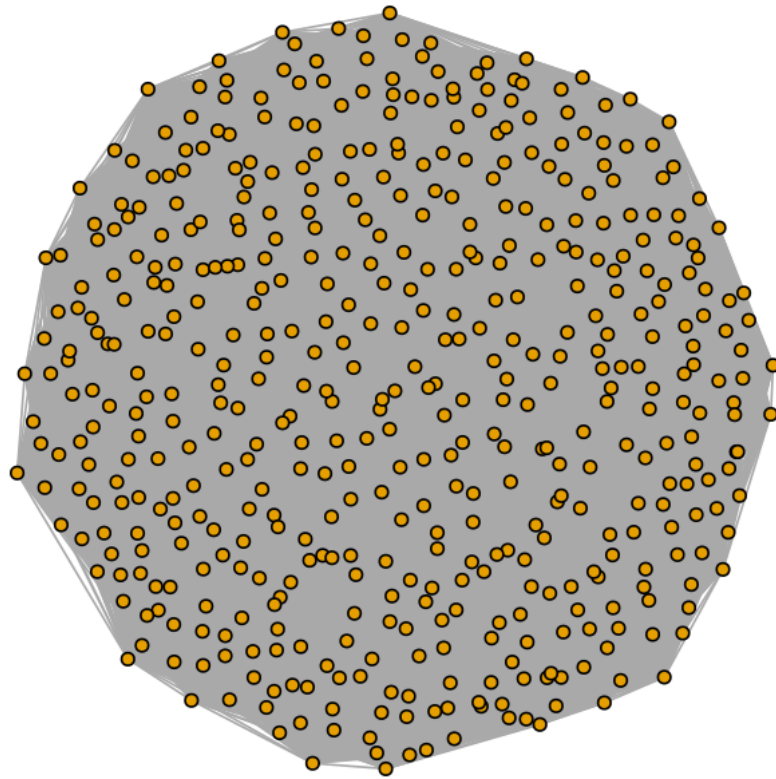
#fname <- matrix(w)
save(w, file = "distance.RData")
```


Unnormalized Distribution of Edge Weights (Daily)

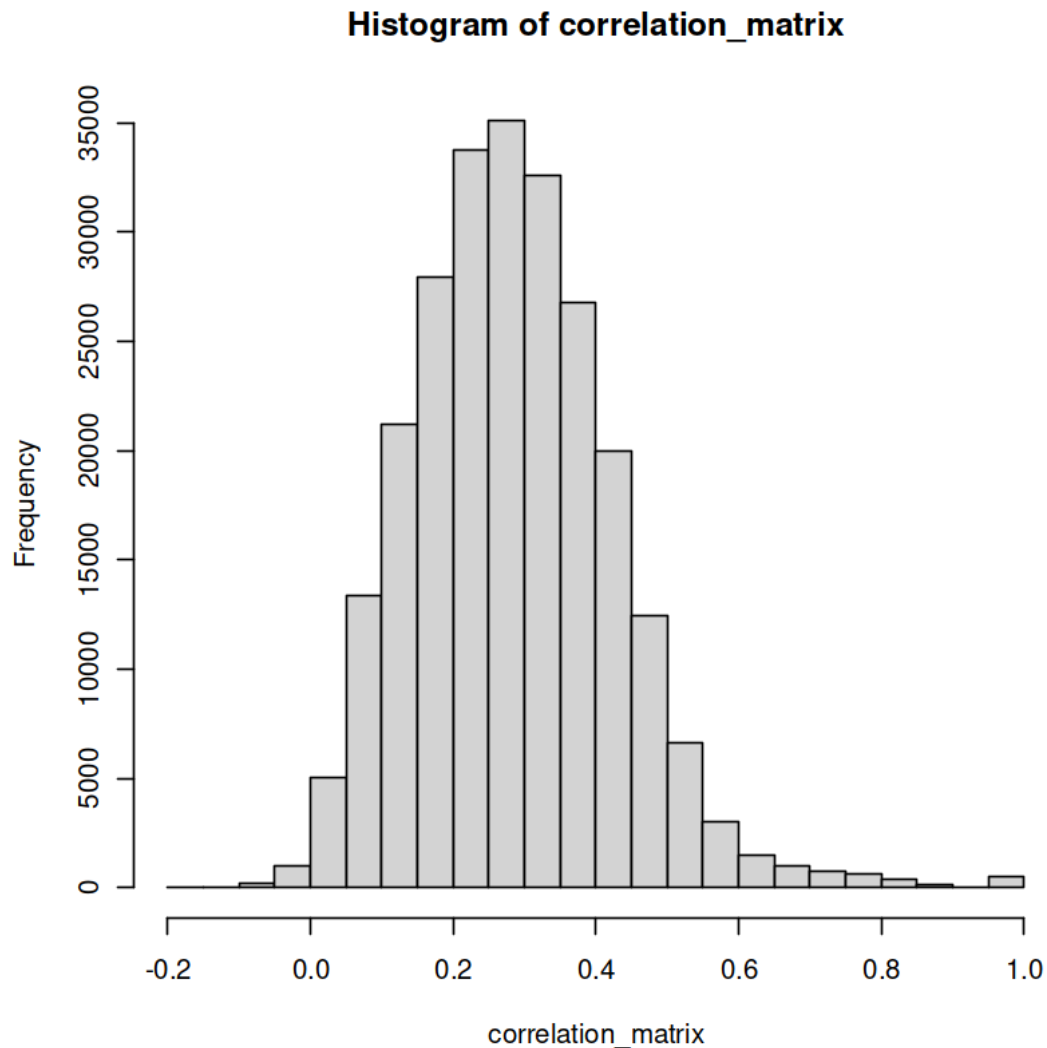


```
[8]: plot(g, vertex.label=NA, vertex.size=3, main="Correlation Graph (Daily)")
```

Correlation Graph (Daily)



```
[9]: hist(correlation_matrix)
```



3 Minimum spanning tree (MST)

3.1 Q3. Plot the MST and color-code the nodes based on sectors.

Each stock can be categorized into a sector, which can be found in Name sector.csv file. Plot the MST and color-code the nodes based on sectors.

Do you see any pattern in the MST? The structures that you find in MST are called Vine clusters. Provide a detailed explanation about the pattern you observe.

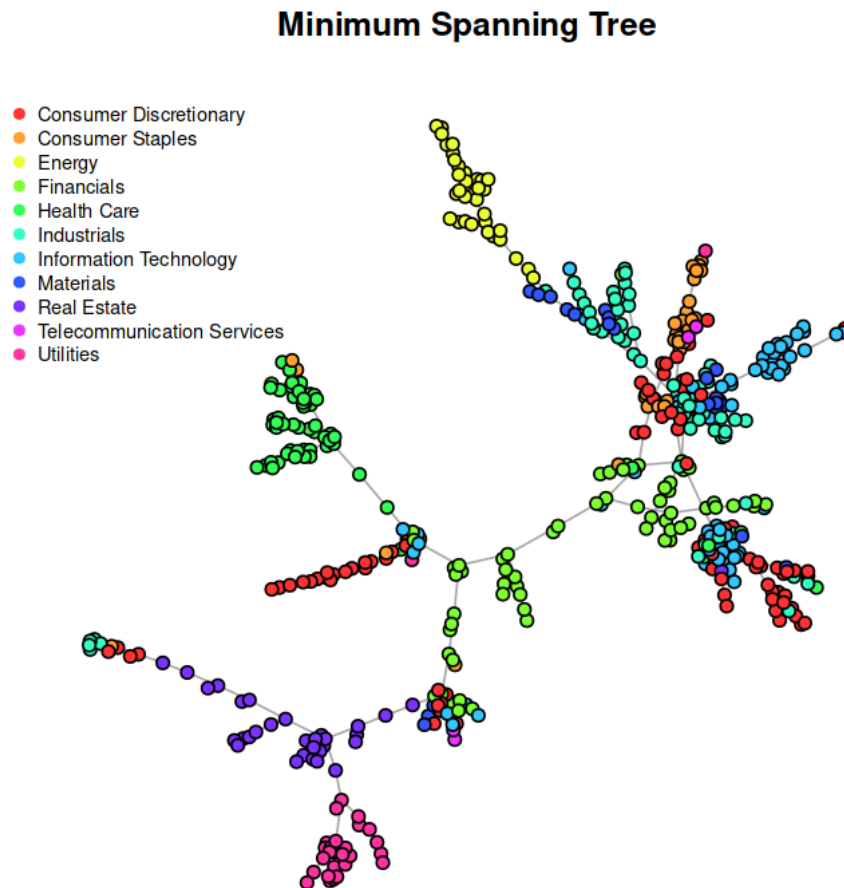
```
[10]: # build MST
mst_g <- mst(g, weights = E(g)$weight)

# plot MST with sector color
colors <- rainbow(num_sector,s = 0.8, v = 1)
plot(mst_g,
```

```

vertex.color = colors[sectors.index],
vertex.size = 3,
vertex.label = NA,
main = "Minimum Spanning Tree")
legend("topleft",
      legend = levels(sectors.set),
      col = colors, pch=21, pt.bg=colors,
      cex=0.7, bty = "n")

```



Answer: The plot illustrates a branching pattern where most nodes within a branch share similar colors, representing the same sectors. The MST highlights the most crucial connections between individuals or groups in the network, revealing the most significant relationships. For example, the Energy and the Materials sectors share a branch of the tree, reflecting their close relationship. Similarly, stocks from the Information Technology and Financials tend to mix in a clump of nodes with other sectors, as they are often considered shared services. Stocks from the Real Estate and the Utilities sectors are also located in close proximity.

3.2 Q4. Run a community detection algorithm (for example walktrap) on the MST obtained above.

Run a community detection algorithm (for example walktrap) on the MST obtained above.

Plot the communities formed. Compute the homogeneity and completeness of the clustering. (you can use the 'clevr' library in r to compute homogeneity and completeness).

```
[11]: #cluster
wc_daily <- cluster_walktrap(mst_g)
length(wc_daily)
plot(wc_daily, mst_g, vertex.size=3, vertex.label=NA,
     vertex.color=sectors.index, main="MST with community")
legend("topleft",
      legend = levels(sectors.set),
      col = colors, pch=21, pt.bg=colors,
      cex=0.7, bty = "n")

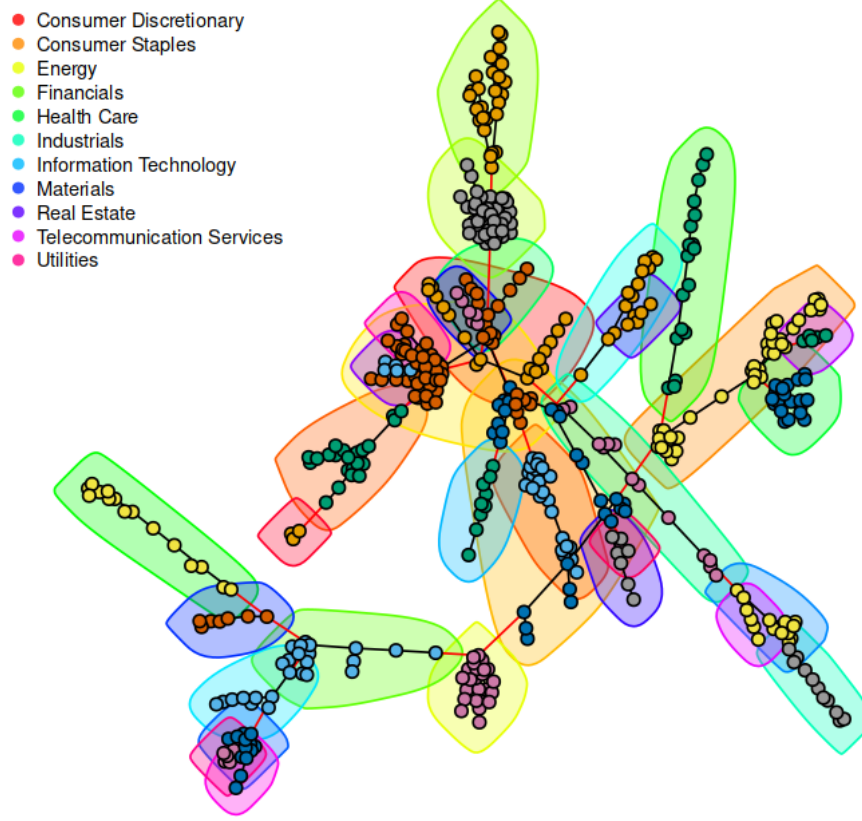
print(paste("Homogeneity: ", homogeneity(sectors.index, wc_daily$membership)))
print(paste("Completeness: ", completeness(sectors.index, wc_daily$membership)))

#get metrics
#true<- c(sectors.index)
#pred <- c(wc_daily$membership)
#homogeneity(true, pred)
#completeness(true,pred)
```

33

```
[1] "Homogeneity:  0.682644648161366"
[1] "Completeness: 0.479284479244588"
```

MST with community



4 Sector clustering in MST's

In order to evaluate the performance of the methods we define the following metric $\alpha = \frac{1}{|V|} \sum_{v_i \in V} P(v_i \in S_i)$, where S_i is the sector of node i .

Define

$$P(v_i \in S_i) = \frac{|Q_i|}{|N_i|}$$

where Q_i is the set of neighbors of node i that belong to the same sector as node i and N_i is the set of neighbors of node i .

Compare α with the case where $P(v_i \in S_i) = \frac{|S_i|}{|V|}$

4.1 Q5. Report the value of α for the above two cases and provide an interpretation for the difference.

predict sector or unknown stock

```
[12]: V <- vcount(mst_g)
P1 <- 0
P2 <- 0

Si <- table(sectors.index)

for (vi in seq_len(V)){
  # get ni
  ni <- neighbors(mst_g, vi) # neighbors of vi
  # get qi -- ni with same sector as vi
  if (length(ni) > 0)
  {
    current_sector = sectors.index[vi]
    qi <- intersect(ni, which(sectors.index == current_sector))
    P1 <- P1 + length(qi)/length(ni)
  }
  # get si - sector of vi
  si <- Si[current_sector]
  #print(paste(current_sector, si, vi))
  P2 <- P2 + si/V
}

alpha1 <- P1/V
alpha2 <- P2/V

print(sprintf("Alpha (Daily) value for method 1 is: %s",alpha1))
print(sprintf("Alpha (Daily) value for method 2 is: %s",alpha2))

cat ("P1: ", P1, "\n")
cat ("P2: ", P2)
```

```
[1] "Alpha (Daily) value for method 1 is: 0.828930077530676"
[1] "Alpha (Daily) value for method 2 is: 0.114188070612532"
P1: 409.4915
P2: 56.40891
```

Answer: In the context of the stock market, the first alpha, using $\frac{|Q_i|}{|N_i|}$, considered only the neighbors of a stock within the same sector. Predictions based on this measure are more restrictive and tend to follow localized trends. In contrast, the second alpha, using $\frac{|S_i|}{|V|}$, considered the overall market. Predictions using this measure are more general, and reflect broader, global trends. Alpha using method 1 is higher than the alpha using method 2, indicating that the first method is more accurate in predicting the sector of an unknown stock.

5 Correlation graphs for weekly data

5.1 Q6. Repeat questions 2,3,4,5 on the WEEKLY data.

weekly

```
[13]: # load file with weekly data
# sample the stock data weekly on Mondays
# If there is a holiday on a Monday, we ignore that week.
path = "finance_data/data/"
out.file<-" "
file.names <- dir(path, pattern = ".csv")
sectors_w.table <- read.table("finance_data/Name_sector.csv",header=TRUE,
  ↪sep=",", stringsAsFactors=TRUE)
sectors_w = c()
sectors_w.names <- c()
weekly_m <- matrix(nrow=0, ncol=143)
for(i in 1:length(file.names)){
  file <- read.table(paste("finance_data/data/",file.names[i],sep =
  ↪""),header=TRUE, sep=",", stringsAsFactors=FALSE)

  Date_data<- file$Date
  weekdays_data <-weekdays(as.Date(Date_data)) #this gives weekday name
  weekly_m_m <- file[weekdays_data == "Monday", 'Close'] #select data where
  ↪week day is monday
  #print (dim(weekly_m))
  #print(length(weekly_m_m))
  #print(dim(matrix(weekly_m_m)))
  #print(dim(t(matrix(weekly_m_m))))
  if(length(weekly_m_m) == 143){
    weekly_m <- rbind(weekly_m, matrix(weekly_m_m, nrow=1, ncol=143))
    mystr <- substr(file.names[i], 1, nchar(file.names[i])-4)
    sector <- sectors_w.table$Sector[which(sectors_w.table$Symbol == mystr)]
#     print(sector)
#     sectors = c(sectors, as.factor(sector))
    sectors_w.names = c(sectors_w.names, toString(sector))
  }
}
com_num = length(sectors_w.names)
data_num = ncol(weekly_m)
sector_w.set<-as.factor(sectors_w.names)
sectors_w.index<-as.numeric(sector_w.set)
num_sector <- length(unique(sectors_w.index))
table(sector_w.set)

# 2014/5/1 - 2017/5/12
# 143 Mondays
```



```
sector_w.set
  Consumer Discretionary      Consumer Staples
                        84                        35
        Energy                Financials
        34                    62
    Health Care                Industrials
        60                    64
Information Technology        Materials
        66                    24
    Real Estate Telecommunication Services
        31                        4
        Utilities
        28
```

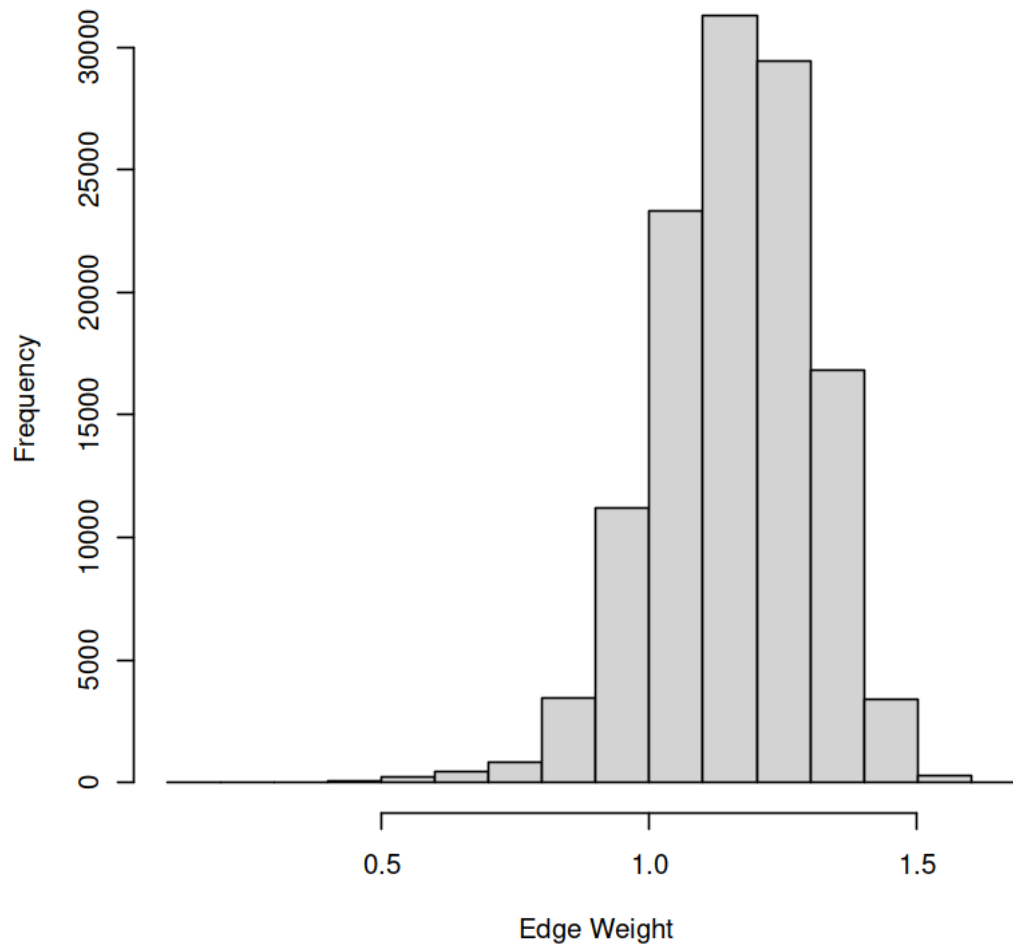
```
[14]: # weekly_m 492x143
week_m_cor <- cal_cor(weekly_m)
week_m_d <- sqrt(2 * (1 - week_m_cor)) #check formula from question pdf

week_m_g<-graph_from_adjacency_matrix(week_m_d, mode ="upper", weighted = TRUE,
→diag = TRUE,
  add.colnames = NULL, add.rownames = NA)

hist(E(week_m_g)$weight, main = "Unnormalized Distribution of Edge Weights"
→(Weekly)", xlab = "Edge Weight")
```

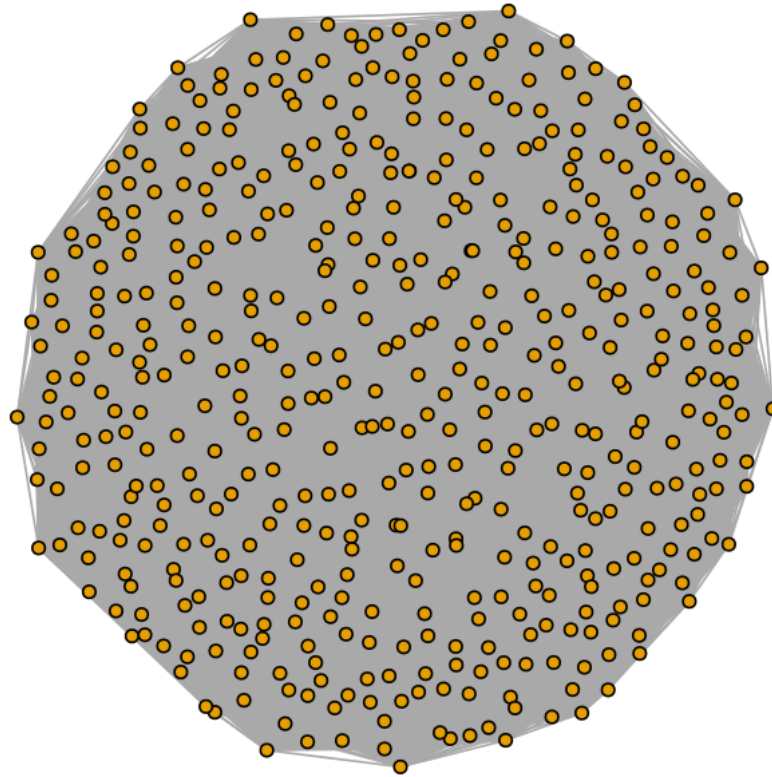
```
[1] "cal cor n x m: 492 143"
```

Unnormalized Distribution of Edge Weights (Weekly)



```
[15]: plot(week_m_g, vertex.label=NA, vertex.size=3, main="Correlation Graph (Weekly)")
```

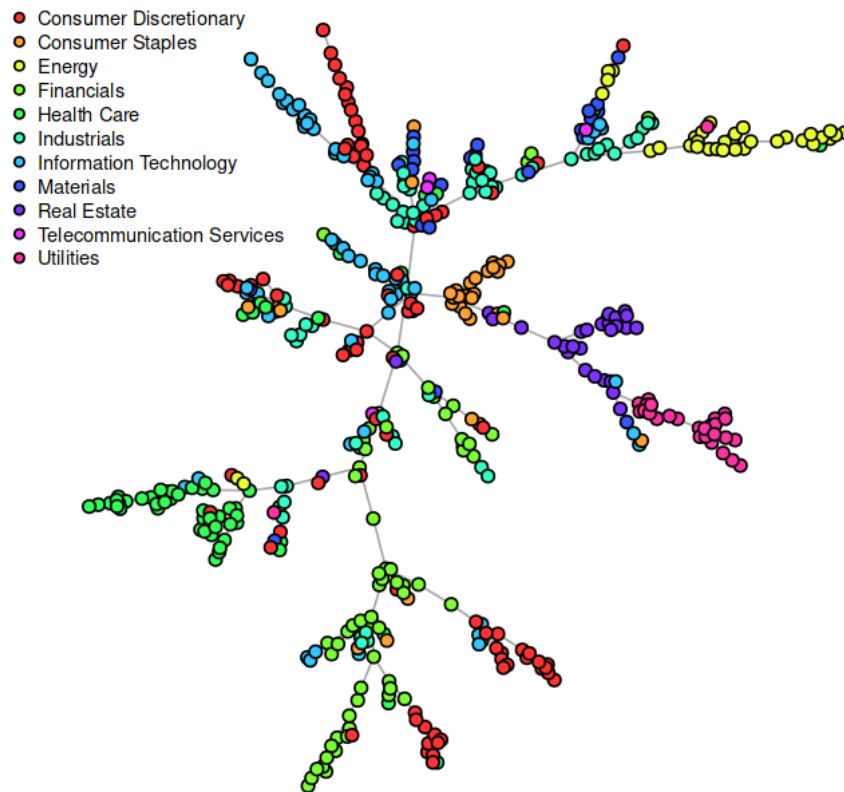
Correlation Graph (Weekly)



```
[16]: mst_week_m <- mst(week_m_g, weights = E(week_m_g)$weight) #use the mst function

colbar <- rainbow(num_sector,s = 0.8, v = 1)
plot(mst_week_m, vertex.size=3, vertex.label=NA, vertex.color=colbar[sectors_w.
  ↪index],
      main = "Minimum Spanning Tree (Weekly)")
legend('topleft',legend=levels(sector_w.set),pch=21, pt.bg=colbar, cex=0.7, bty="n"
  ↪= "n")
```

Minimum Spanning Tree (Weekly)



Q3 Answer: Although we still have the tree branches, the sectors merge more as we observe fewer single-color branches than in the daily MST. The sectors are more intermixed when sampling weekly sampling than when sampling daily. Several branches have lone-nodes of different colors, signaling possible misclassification. The MST is less clear (in terms of sector separation) and less accurate (misclassification) in the weekly sampling.

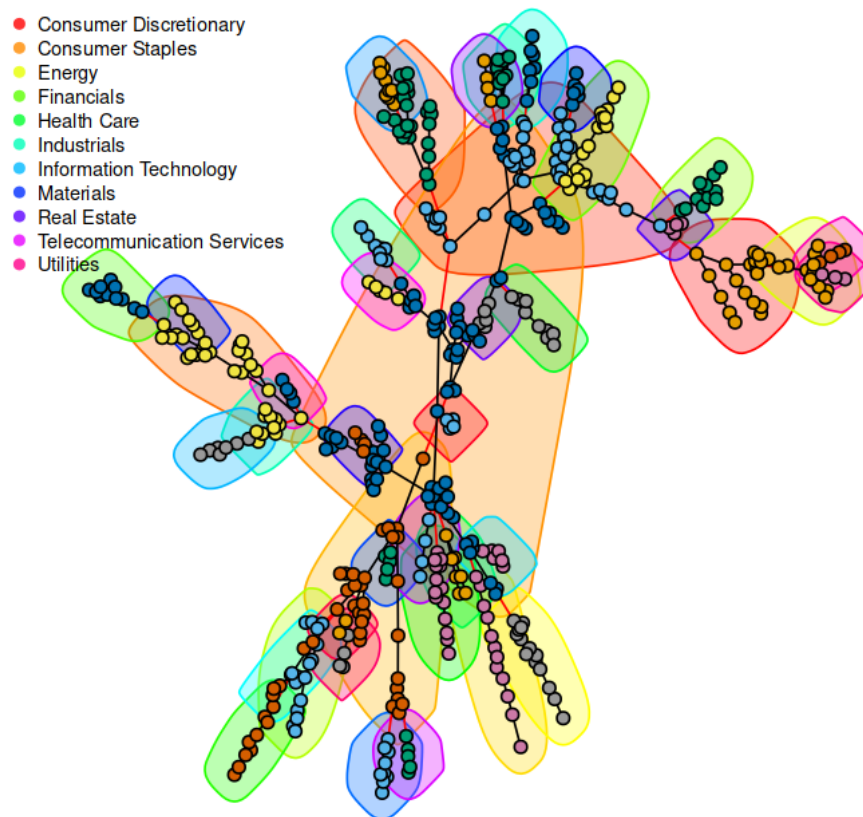
```
[17]: #cluster
wc_weekly <- cluster_walktrap(mst_week_m) #you can use walktrap.community.␣
→ensure that same algo is used for daily, monthly, weekly
length(wc_weekly)
plot(wc_weekly, mst_week_m, vertex.size=3, vertex.label=NA,
     vertex.color=sectors_w.index, main="MST with community (Weekly)")
legend("topleft",
      legend = levels(sectors_w.set),
      col = colors, pch=21, pt.bg=colors,
      cex=0.7, bty = "n")
```

```
print(paste("Homogeneity: ", homogeneity(sectors_w.index, wc_weekly$membership)))
print(paste("Completeness: ", completeness(sectors_w.index,
↪wc_weekly$membership)))
```

42

```
[1] "Homogeneity: 0.581123705715172"
[1] "Completeness: 0.390043529189446"
```

MST with community (Weekly)



```
[18]: #get metrics
true<- c(sectors_w.index)
pred <- c(wc_weekly$membership)
homogeneity(true, pred)
completeness(true,pred)
```

0.581123705715172

0.390043529189446

```
[19]: V <- vcount(mst_week_m)
P1 <- 0
P2 <- 0

Si <- table(sectors_w.index)

for (vi in seq_len(V)){
  # get ni
  ni <- neighbors(mst_week_m, vi) # neighbors of vi
  # get qi -- ni with same sector as vi
  if (length(ni) > 0)
  {
    current_sector = sectors_w.index[vi]
    qi <- intersect(ni, which(sectors_w.index == current_sector))
    P1 <- P1 + length(qi)/length(ni)
  }
  # get si - sector of vi
  si <- Si[current_sector]
  #print(paste(current_sector, si, vi))
  P2 <- P2 + si/V
}

alpha1 <- P1/V
alpha2 <- P2/V

print(sprintf("Alpha (Weekly) value for method 1 is: %s",alpha1))
print(sprintf("Alpha (Weekly) value for method 2 is: %s",alpha2))

cat ("P1: ", P1, "\n")
cat ("P2: ", P2)
```

```
[1] "Alpha (Weekly) value for method 1 is: 0.743957284810943"
```

```
[1] "Alpha (Weekly) value for method 2 is: 0.114308612598321"
```

```
P1: 366.027
```

```
P2: 56.23984
```

Q5 Answer: The alpha value for method 1 is 0.7440, and the alpha value for method 2 is 0.1143. The alpha value for method 1 is higher than the alpha value for method 2, indicating that the first method is more accurate in predicting the sector of an unknown stock.

6 Correlation graphs for MONTHLY data

6.1 Q7. Repeat questions 2,3,4,5 on the MONTHLY data.

Monthly

```

[20]: # load file with Monthly data
# sample the stock data Monthly on 15th and
# If there is a holiday on the 15th, we ignore that month.

#dates_num <- #format(#write code here)
#monthly_15_m <- #select data where date is 15

path = "finance_data/data/"
out.file<-" "
file.names <- dir(path, pattern = ".csv")
sectors_m.table <- read.table("finance_data/Name_sector.csv",header=TRUE,
  ↪sep=",", stringsAsFactors=TRUE)
sectors_m = c()
sectors_m.names <- c()
monthly_15_m <- matrix(,nrow=0, ncol=25)
for(i in 1:length(file.names)){
  file <- read.table(paste("finance_data/data/",file.names[i],sep =
  ↪"),header=TRUE, sep=",", stringsAsFactors=FALSE)

  Date_data<- file$Date
  dates_num <- format(as.Date(Date_data), "%d") #format get day from date
  ↪string
  # print(dates_num)
  monthly_m_m <- file[dates_num == "15", 'Close'] #select 15th of month
  #print(dim(monthly_15_m))
  #print(length(monthly_m_m))
  #print(dim(matrix(monthly_m_m)))
  #print(dim(t(matrix(monthly_m_m))))
  if(length(monthly_m_m) == 25){
    monthly_15_m <- rbind(monthly_15_m, matrix(monthly_m_m, nrow=1, ncol=25))
    mystr <- substr(file.names[i], 1, nchar(file.names[i])-4)
    sector <- sectors_m.table$Sector[which(sectors_m.table$Symbol == mystr)]
#     print(sector)
#     sectors = c(sectors, as.factor(sector))
    sectors_m.names = c(sectors_m.names, toString(sector))
  }
}
com_num = length(sectors_m.names)
data_num = ncol(monthly_15_m)
sector_m.set<-as.factor(sectors_m.names)
sectors_m.index<-as.numeric(sector_m.set)
num_sector <- length(unique(sectors_m.index))
table(sector_m.set)

# 2014/5/1 - 2017/5/12
# 25 month with 15th not a holiday
# 1 min 30 sec

```

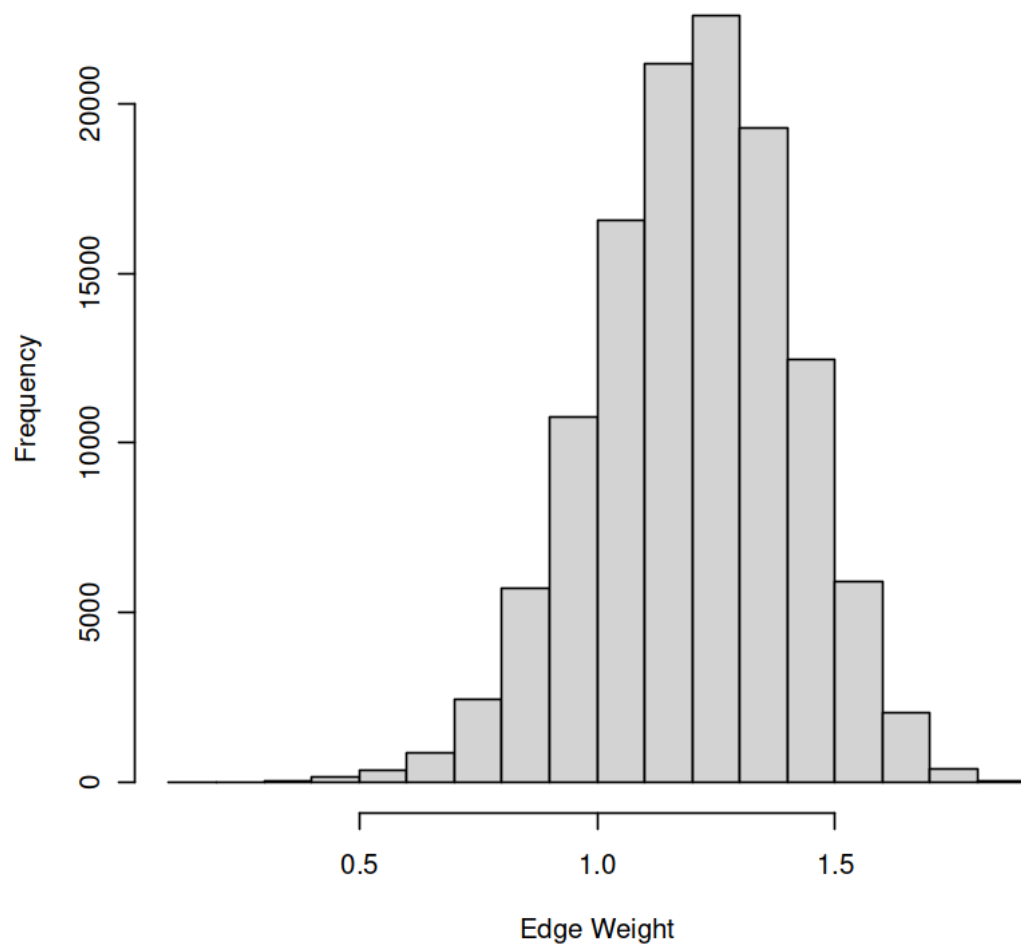
```
sector_m.set
  Consumer Discretionary      Consumer Staples
                        84                        35
                        Energy      Financials
                        34                        62
      Health Care      Industrials
                        60                        64
  Information Technology      Materials
                        66                        24
      Real Estate Telecommunication Services
                        31                        4
      Utilities
                        28
```

```
[21]: # from helper code
# monthly_15_m #492x25
monthly_15_cor <- cal_cor(monthly_15_m )
monthly_15_d <- sqrt(2 * (1 - monthly_15_cor)) #check formula from question pdf
month_15_g<-graph_from_adjacency_matrix(monthly_15_d, mode ="upper", weighted =
→TRUE, diag = TRUE,
  add.colnames = NULL, add.rownames = NA)

hist(E(month_15_g)$weight, main = "Unnormalized Distribution of Edge Weights
→(Monthly 15th)", xlab = "Edge Weight")
```

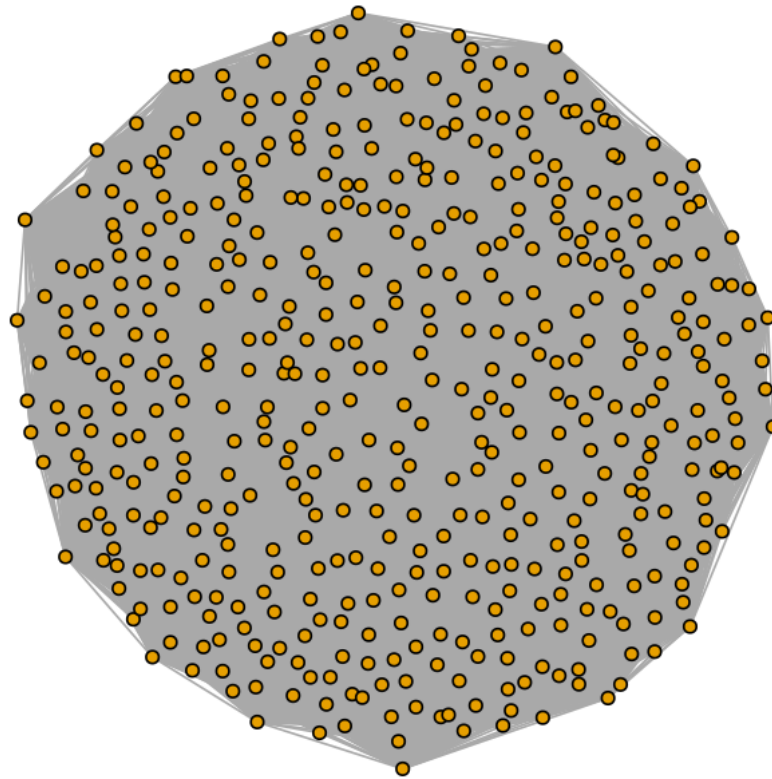
```
[1] "cal cor n x m: 492 25"
```


Unnormalized Distribution of Edge Weights (Monthly 15th)



```
[22]: plot(month_15_g, vertex.label=NA, vertex.size=3, main="Correlation Graph_↵  
↵(Monthly 15th)")
```

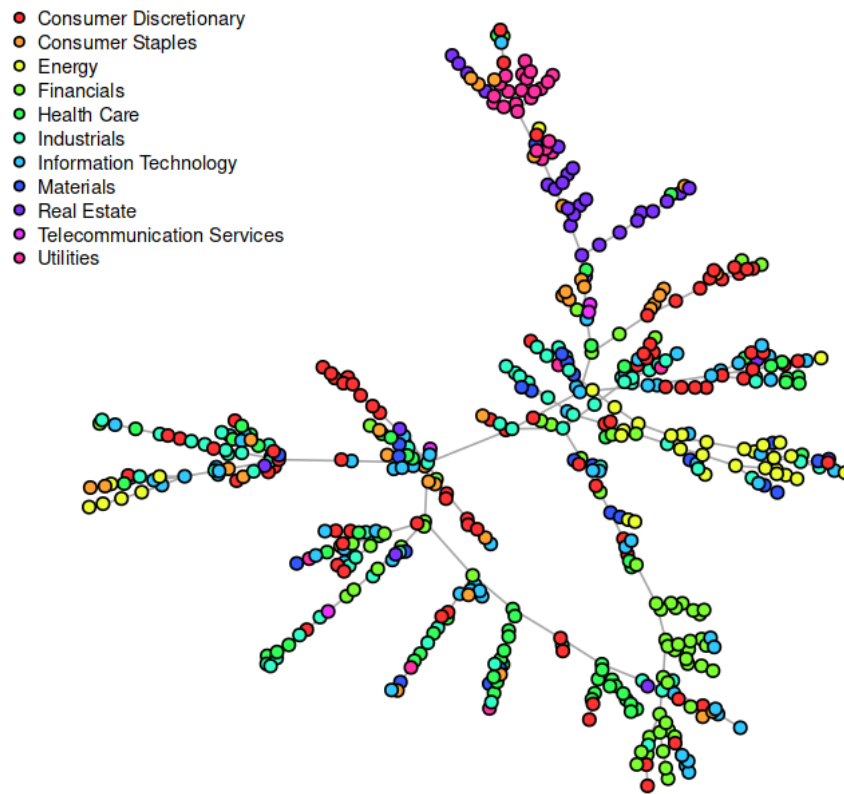
Correlation Graph (Monthly 15th)



```
[23]: mst_month_15 <- mst(month_15_g, weights = E(month_15_g)$weight) #use the mst_
      ↪ function
colbar <- rainbow(num_sector,s = 0.8, v = 1)
plot(mst_month_15, vertex.size=3, vertex.label=NA, vertex.color=colbar[sectors_m.
      ↪ index],
      main = "Minimum Spanning Tree (Monthly 15th)")

legend('topleft',legend=levels(sectors_m.set),pch=21, pt.bg=colbar, cex=0.7, bty_
      ↪ = "n")
```

Minimum Spanning Tree (Monthly 15th)



Q3 Answer: Again for the monthly MST, we still have the tree branches, however, the sectors merge even more as we observe fewer single-color branches. The sectors are more intermixed when sampling monthly than when sampling daily or weekly. Almost every branch includes nodes of different colors (sectors). It no longer follows the same sector / color pattern as the daily MST.

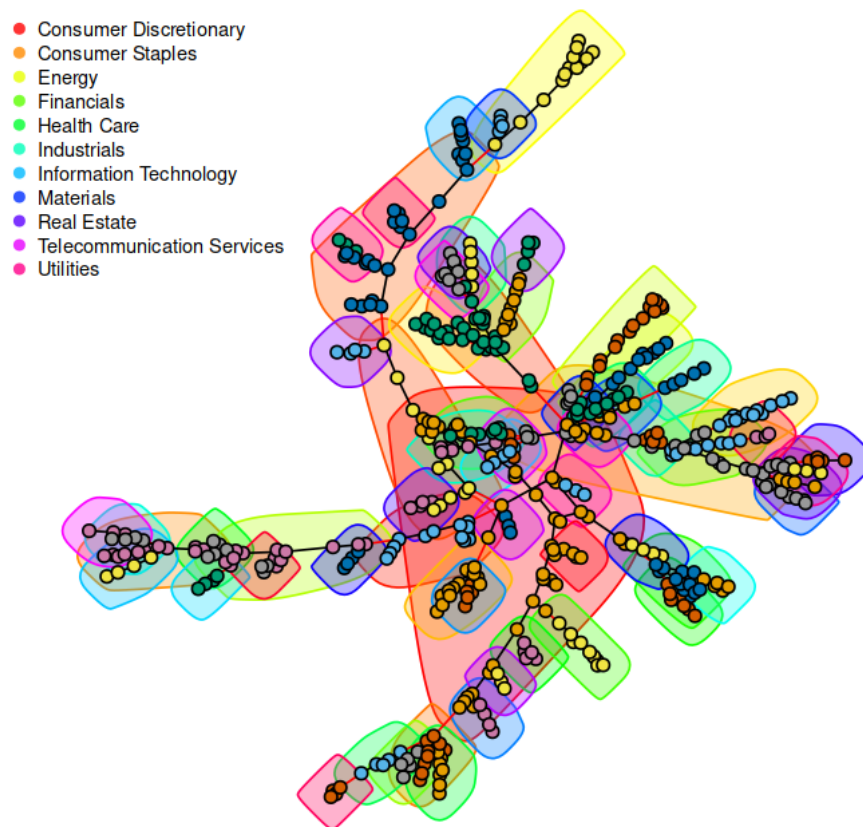
```
[24]: #cluster
wc_monthly <- cluster_walktrap(mst_month_15)
length(wc_monthly)
plot(wc_monthly, mst_month_15, vertex.size=3, vertex.label=NA,
     vertex.color=sectors_m.index, main="MST with community (Monthly 15th)")
legend("topleft",
      legend = levels(sector_m.set),
      col = colors, pch=21, pt.bg=colors,
      cex=0.7, bty = "n")
```

```
print(paste("Homogeneity: ", homogeneity(sectors_m.index,
↪wc_monthly$membership)))
print(paste("Completeness: ", completeness(sectors_m.index,
↪wc_monthly$membership)))
```

65

```
[1] "Homogeneity: 0.479447330844778"
[1] "Completeness: 0.277551241133061"
```

MST with community (Monthly 15th)



```
[25]: #get metrics
true<- c(sectors_m.index)
pred <- c(wc_monthly$membership)
homogeneity(true, pred)
completeness(true,pred)
```

0.479447330844778

0.277551241133061

```
[26]: V <- vcount(mst_month_15)
P1 <- 0
P2 <- 0
alpha1 <- 0
alpha2 <- 0
Si <- table(sectors_m.index)

for (vi in seq_len(V)){
  # get ni
  ni <- neighbors(mst_month_15, vi) # neighbors of vi
  # get qi -- ni with same sector as vi
  if (length(ni) > 0)
  {
    current_sector = sectors_m.index[vi]
    qi <- intersect(ni, which(sectors_m.index == current_sector))
    P1 <- P1 + length(qi)/length(ni)
  }
  # get si - sector of vi
  si <- Si[current_sector]
  #print(paste(current_sector, si, vi))
  P2 <- P2 + si/V
}

alpha1 <- P1/V
alpha2 <- P2/V

print(sprintf("Alpha (Monthly) value for method 1 is: %s",alpha1))
print(sprintf("Alpha (Monthly) value for method 2 is: %s",alpha2))

cat ("P1: ", P1, "\n")
cat ("P2: ", P2)
```

```
[1] "Alpha (Monthly) value for method 1 is: 0.484446380178087"
```

```
[1] "Alpha (Monthly) value for method 2 is: 0.114308612598321"
```

```
P1: 238.3476
```

```
P2: 56.23984
```

Q5 Answer: The alpha value for method 1 is 0.4844, and the alpha value for method 2 is 0.1143. The alpha value for method 1 is higher than the alpha value for method 2, indicating that the first method is still more accurate in predicting the sector of an unknown stock.

6.2 Q8. Compare and analyze all the results of daily data vs weekly data vs monthly data.

What trends do you find? What changes? What remains similar? Give reason for your observations. Which granularity gives the best results when predicting the sector of an unknown stock and why?

	Daily	Weekly	Monthly
m (data point)	765	143	25
Homogeneity	0.6826	0.5811	0.4794
Completeness	0.4793	0.3900	0.2776
Alpha 1	0.8289	0.7440	0.4844
Alpha 2	0.1142	0.1143	0.1143
Clusters	33	42	65

Answer:

Summary of Daily, Weekly, and Monthly Results

Alpha 2 remains similar across all three sampling frequencies, while alpha 1 decreases as the sampling frequency decreases. Sampling weekly and monthly is less accurate than sampling daily with the closed price. Monthly sampling provides less information (i.e. fewer data points), and thus the observed trends are less clear. Both Homogeneity and Completeness are lower for weekly and monthly sampling because the stock market is less volatile, and the trends are less clear. Daily sampling is the most accurate because the stock market is the most volatile, and the trends are more defined, as evident in the high alpha 1 value. The number of clusters increases as the sampling frequency decreases, indicating that the data is more segmented and less clustered. This is consistent with the segmentation of the data, as shown in the monthly MST plots. Note that the monthly number of clusters (65) is almost double of the daily number (33).

Daily sampling is the best for predicting the sector of an unknown stock because it is the most accurate and contains the most information.

Part II

Let's Help Santa!

1 Download the Data

The dataset contains pairwise traveling time statistics between most pairs of points in the Los Angeles area. Points on the map are represented by unique IDs. To understand the correspondence between map IDs and areas, download Geo Boundaries file from the same website. This file contains latitudes and longitudes of the corners of the polygons circumscribing each area. To be specific, if an area is represented by a polygon with 5 corners, then you have a 5×2 matrix of the latitudes and longitudes, each row of which represents latitude and longitude of one corner.

```
[ ]: with open('los_angeles_censustracts.json') as f:
      census_tracts = json.loads(f.readline())
```

time: 156 ms (started: 2025-08-23 03:18:09 +00:00)

```
[ ]: census_tracts['features'][0]['properties']
```

```
[ ]: {'MOVEMENT_ID': '1', 'TRACT': '480302', 'DISPLAY_NAME': 'Census Tract 480302'}
```

time: 1.92 ms (started: 2025-08-23 03:18:11 +00:00)

```
[ ]: np.array(census_tracts['features'][0]['geometry']['coordinates'][0]).  
      ↪mean(axis=0) # [longitude, latitude]
```

```
[ ]: array([-118.12053321,  34.10309557])
```

time: 3.15 ms (started: 2025-08-23 03:18:14 +00:00)

```
[ ]: display_names = dict()  
      coordinates = dict()  
  
      for area in census_tracts['features']:  
          id = int(area['properties']['MOVEMENT_ID'])  
          display_name = area['properties']['DISPLAY_NAME']  
          display_names[id] = display_name  
  
          a = area['geometry']['coordinates'][0]  
          coordinates[id] = np.array(a if type(a[0][0]) == float else a[0]).  
          ↪mean(axis=0)
```

time: 106 ms (started: 2025-08-23 03:18:15 +00:00)

```
[ ]: for i in range(5):  
      key = i + 1  
      print('{}: {}, {}'.format(key, display_names[key], coordinates[key]))
```

```
1: Census Tract 480302, [-118.12053321  34.10309557]  
2: Census Tract 480303, [-118.13785063  34.09645121]  
3: Census Tract 480304, [-118.13138209  34.09626386]  
4: Census Tract 480400, [-118.13224544  34.10349303]  
5: Census Tract 480500, [-118.14492317  34.0986815 ]  
time: 1.59 ms (started: 2025-08-23 03:18:19 +00:00)
```

2 Build Your Graph

2.1 Q9. Report the number of nodes and edges in G.

```
[ ]: g = Graph(directed=False)  
      g.add_vertices(len(display_names))  
      g.vs['display_name'] = list(display_names.values()) # index = id - 1  
      # Add the centroid coordinates of each polygon region (a 2-D vector) as an  
      ↪attribute to the corresponding vertex.  
      g.vs['coordinates'] = list(coordinates.values())
```

time: 1.29 ms (started: 2025-08-23 03:18:21 +00:00)

```
[ ]: month_filter = {12} # for monthly aggregate data of 4th quarter, we can filter  
      ↪data based off of only December
```

```

edges = []
weights = []

with open('los_angeles-censustracts-2019-4-All-MonthlyAggregate.csv') as f:
    f.readline() # skip the first line

    while True:
        line = f.readline()
        if line == '':
            break # end of file

        vals = line.strip().split(',')

        # read edge info
        src, dest, month, dist = int(vals[0]), int(vals[1]), int(vals[2]), float(vals[3])

        # if data is not relevant, skip it
        if month not in month_filter:
            continue

        edges.append((src - 1, dest - 1))
        weights.append(dist)

```

time: 9.19 s (started: 2025-08-23 03:18:23 +00:00)

```

[ ]: # the graph's weight is the mean travel time between source and dest from csv
g.add_edges(edges)
g.es['weight'] = weights
del edges, weights

```

time: 1.38 s (started: 2025-08-23 03:18:39 +00:00)

```

[ ]: print(len(g.vs), len(g.es))

```

2716 1692450

time: 573 µs (started: 2025-08-23 03:18:43 +00:00)

```

[ ]: # keep only the giant connected component
components = g.components()
gcc = max(components, key=len)
vs_to_delete = [i for i in range(len(g.vs)) if i not in gcc]
g.delete_vertices(vs_to_delete)

# remove duplicate edges (merge duplicate edges by averaging their weights)
g = g.simplify(combine_edges=dict(weight='mean')) # combine duplicate edges

```

time: 2.39 s (started: 2025-08-23 03:18:47 +00:00)


```
[ ]: print(len(g.vs), len(g.es))
```

2649 1003858

time: 664 µs (started: 2025-08-23 03:18:53 +00:00)

Answer: There are 2,649 nodes and 1,003,858 edges.

3 Traveling Salesman Problem

3.1 Q10. Build a minimum spanning tree (MST) of graph G.

Report the street addresses near the two endpoints (the centroid locations) of a few edges. Are the results intuitive?

```
[ ]: mst = g.spanning_tree(weights=g.es['weight'])
```

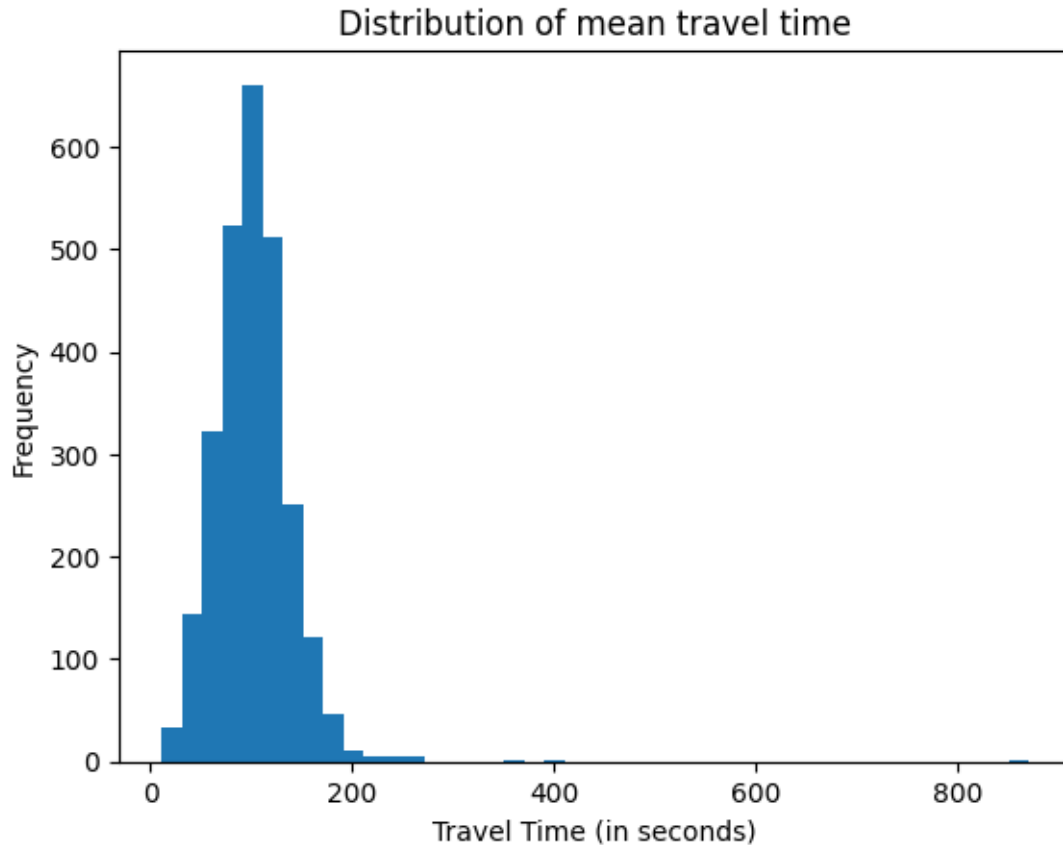
time: 632 ms (started: 2025-08-23 03:19:00 +00:00)

```
[ ]: edf = mst.get_edge_dataframe()
     edf.head()
```

```
[ ]:
      source target  weight
edge ID
0         0      2  129.765
1         0     13  118.335
2         1      2   90.235
3         1      3  126.475
4         1      9  125.675
```

time: 37.4 ms (started: 2025-08-23 03:19:04 +00:00)

```
[ ]: import matplotlib.pyplot as plt
     weights = mst.es['weight']
     binwidth = 20
     bins = np.arange(min(weights), max(weights) + binwidth, binwidth)
     plt.hist(weights, bins=bins)
     plt.xlabel('Travel Time (in seconds)')
     plt.ylabel('Frequency')
     plt.title('Distribution of mean travel time')
     plt.show()
```



time: 242 ms (started: 2025-08-23 03:19:15 +00:00)

```
[ ]: for i, e in enumerate(mst.es):
      print('Distance in miles: {:.3f}\tTime taken: {:.1f} sec'.format(linalg.
      ↪norm(mst.vs[e.source]['coordinates'] - mst.vs[e.target]['coordinates'])* 69,
      ↪e['weight']))
      if i > 10:
          break
```

Distance in miles: 0.885	Time taken: 129.8 sec
Distance in miles: 0.570	Time taken: 118.3 sec
Distance in miles: 0.447	Time taken: 90.2 sec
Distance in miles: 0.621	Time taken: 126.5 sec
Distance in miles: 0.812	Time taken: 125.7 sec
Distance in miles: 0.618	Time taken: 119.9 sec
Distance in miles: 0.936	Time taken: 125.2 sec
Distance in miles: 0.412	Time taken: 91.8 sec
Distance in miles: 0.256	Time taken: 60.9 sec
Distance in miles: 0.204	Time taken: 87.1 sec
Distance in miles: 0.620	Time taken: 110.9 sec
Distance in miles: 0.493	Time taken: 162.3 sec

time: 1.46 ms (started: 2025-08-23 03:19:23 +00:00)

```
[ ]: # printing addresses, coordinates
# add the distance just in case
import folium
from geopy.geocoders import Nominatim
geolocator = Nominatim(user_agent="lacounty")
start=mst.vs[0]['coordinates']
m = folium.Map(location=(start[1], start[0]), zoom_start=14, width="70%")
for i, e in enumerate(mst.es):
    try:
        src=mst.vs[e.source]['coordinates']
        dst=mst.vs[e.target]['coordinates']
        location1 = geolocator.reverse((src[1], src[0]), timeout=None)
        folium.Marker(location=[src[1], src[0]], popup=f"src {i} - {location1.
→address}", icon=folium.Icon(color="green")).add_to(m)
        location2 = geolocator.reverse((dst[1], dst[0]), timeout=None)
        folium.Marker(location=[dst[1], dst[0]], popup=f"dst {i} - {location2.
→address}", icon=folium.Icon(color="blue")).add_to(m)
        print(location1.address)
        print(location2.address)
        print(f"\t({e.source}, {e.target}) - Source address: {src}, Target address:
→{dst}")
        mile=linalg.norm(src - dst)* 69 # mile = degree * 69
        time=e['weight'] # in sec
        print('\tDistance in miles: {:.3f}\tTime taken: {:.1f}\tAvg Mph: {:.1f}'.
→format(mile, time, mile/time*60*60))
    except:
        continue
if i > 10:
    break
m
```

Sacred Heart Retreat House - Saint Joseph Campus, 507, North Granada Avenue,
Alhambra, Los Angeles County, California, 91801, United States
269, Woodward Avenue, Alhambra, Los Angeles County, California, 91801, United
States

(0, 2) - Source address: [-118.12053321 34.10309557], Target address:
[-118.13138209 34.09626386]

Distance in miles: 0.885 Time taken: 129.8 Avg Mph: 24.5

Sacred Heart Retreat House - Saint Joseph Campus, 507, North Granada Avenue,
Alhambra, Los Angeles County, California, 91801, United States
320, Cordova Street, Alhambra, Los Angeles County, California, 91801, United
States

(0, 13) - Source address: [-118.12053321 34.10309557], Target address:
[-118.11656383 34.09585388]

Distance in miles: 0.570 Time taken: 118.3 Avg Mph: 17.3

402, North Marguerita Avenue, Alhambra, Los Angeles County, California, 91801,
United States

269, Woodward Avenue, Alhambra, Los Angeles County, California, 91801, United States

(1, 2) - Source address: [-118.13785063 34.09645121], Target address: [-118.13138209 34.09626386]

Distance in miles: 0.447 Time taken: 90.2 Avg Mph: 17.8

402, North Marguerita Avenue, Alhambra, Los Angeles County, California, 91801, United States

926, North Garfield Avenue, Alhambra, Los Angeles County, California, 91801, United States

(1, 3) - Source address: [-118.13785063 34.09645121], Target address: [-118.13224544 34.10349303]

Distance in miles: 0.621 Time taken: 126.5 Avg Mph: 17.7

402, North Marguerita Avenue, Alhambra, Los Angeles County, California, 91801, United States

Orange Street, Alhambra, Los Angeles County, California, 91803, United States

(1, 9) - Source address: [-118.13785063 34.09645121], Target address: [-118.14184446 34.08538654]

Distance in miles: 0.812 Time taken: 125.7 Avg Mph: 23.3

269, Woodward Avenue, Alhambra, Los Angeles County, California, 91801, United States

547, South 5th Street, Alhambra, Los Angeles County, California, 91801, United States

(2, 12) - Source address: [-118.13138209 34.09626386], Target address: [-118.12911933 34.08759475]

Distance in miles: 0.618 Time taken: 119.9 Avg Mph: 18.6

926, North Garfield Avenue, Alhambra, Los Angeles County, California, 91801, United States

1992, West Alhambra Road, Alhambra, Los Angeles County, California, 91801, United States

(3, 4) - Source address: [-118.13224544 34.10349303], Target address: [-118.14492317 34.0986815]

Distance in miles: 0.936 Time taken: 125.2 Avg Mph: 26.9

1992, West Alhambra Road, Alhambra, Los Angeles County, California, 91801, United States

2450, West Grand Avenue, Alhambra, Los Angeles County, California, 91801, United States

(4, 8) - Source address: [-118.14492317 34.0986815], Target address: [-118.15023891 34.09595766]

Distance in miles: 0.412 Time taken: 91.8 Avg Mph: 16.2

2666, West Alhambra Road, Alhambra, Los Angeles County, California, 91801, United States

2450, West Grand Avenue, Alhambra, Los Angeles County, California, 91801, United States

(5, 8) - Source address: [-118.1528085 34.098628], Target address: [-118.15023891 34.09595766]

Distance in miles: 0.256 Time taken: 60.9 Avg Mph: 15.1

2666, West Alhambra Road, Alhambra, Los Angeles County, California, 91801, United States

4 TSP Approximation

4.1 Q11. Determine what percentage of triangles in the graph (sets of 3 points on the map) satisfy the triangle inequality.

You do not need to inspect all triangles, you can just estimate by random sampling of 1000 triangles.

Now, we want to find an approximation solution for the traveling salesman problem (TSP) on G. Apply the 1-approximate algorithm described in the class. Inspect the sequence of street addresses visited on the map and see if the results are intuitive.

4.1.1 Check Triangle Inequality

```
[ ]: def is_triangle(sides):  
  
    sort = sorted(sides)  
  
    if (sort[0] + sort[1]) > sort[2]:  
        return True  
    else:  
        return False  
  
print('Test:', is_triangle([1,4,2]), is_triangle([3,5,3]))
```

Test: False True

time: 859 μ s (started: 2025-08-23 03:20:15 +00:00)

```
[ ]: def sample_triangles1(g, n_samples):  
    #result will have tuple of the the vertices of a triangle  
    triangles = g.cliques(min=3, max=3)  
    return random.sample(triangles, n_samples)
```

time: 633 μ s (started: 2025-08-23 03:20:17 +00:00)

```
[ ]: n_samples = 1000  
triangles = 0  
sampled = sample_triangles1(g, n_samples)  
  
for triangle_vs in sampled:  
  
    edge_lengths = []  
  
    for i in range(3):  
        for j in range(i):  
            edge_lengths.append(min(g.es.select(_between=([triangle_vs[i]],  
→[triangle_vs[j]]))['weight']))  
  
    triangles += is_triangle(edge_lengths)
```

```
print('Triangle inequality holds by {}%'.format(round(100.0 * triangles /
↪n_samples, 1)))
# 7 min for 1000
```

Triangle inequality holds by 93.7%

time: 6min 37s (started: 2025-08-23 03:20:20 +00:00)

```
[ ]: def preorder_traversal(root, root_parent=None):
    result = []

    if root != None:
        result.append(root)

        for neighbor in root.neighbors(): # Assuming 'out' for directed edges
↪(parent -> child)
            if neighbor != root_parent: # Avoid infinite loops in case of
↪non-tree graphs
                result.extend(preorder_traversal(neighbor, root))

    return result
```

time: 909 μs (started: 2025-08-23 03:26:59 +00:00)

4.1.2 Preorder Traversal Demo

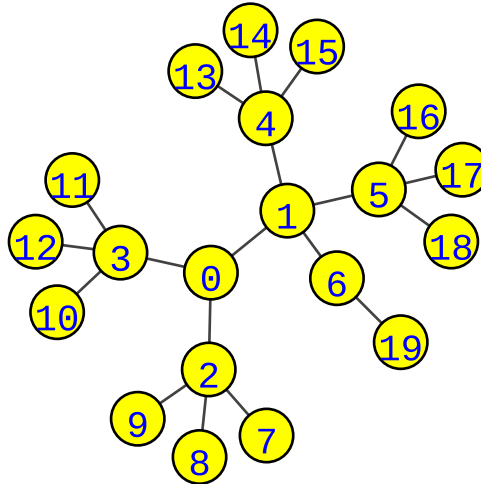
```
[ ]: !pip install --upgrade pip
!pip install pycairo
!pip install igraph
!pip install cairocffi
import cairocffi
import igraph
```

```
[ ]: t = Graph.Tree(20, 3)

visual_style = {}
visual_style["vertex_color"] = 'yellow'
visual_style["vertex_label"] = [v.index for v in t.vs]
visual_style["vertex_label_color"] = 'blue'
visual_style["bbox"] = (200, 200)

plot(t, **visual_style)
```

```
[ ]:
```



time: 126 ms (started: 2025-08-23 03:27:41 +00:00)

```
[ ]: print([v.index for v in preorder_traversal(t.vs[0])])
```

[0, 1, 4, 13, 14, 15, 5, 16, 17, 18, 6, 19, 2, 7, 8, 9, 3, 10, 11, 12]

time: 719 μs (started: 2025-08-23 03:27:45 +00:00)

4.1.3 Actual Data

```
[ ]: #double up on edges to make a eulerian graph
eulerian_g = Graph(directed=False)
eulerian_g.add_vertices(mst.vcount())
for edge in mst.es:
    source, target = g.vs[edge.source].index, g.vs[edge.target].index
    eulerian_g.add_edge(source, target)
    eulerian_g.add_edge(source, target)
```

time: 649 ms (started: 2025-08-23 03:27:49 +00:00)

```
[ ]: #eulerian path

stack = [0]
eulerian_path = []

while stack:
    v = stack[-1]
    if eulerian_g.degree()[v] > 0:
        for i in eulerian_g.neighbors(v):
            eid = eulerian_g.get_eid(v, i)
            eulerian_g.delete_edges(eid)
            stack.append(i)
```



```

        break
    else:
        eulerian_path.append(stack.pop())

```

time: 1.21 s (started: 2025-08-23 03:27:54 +00:00)

```
[ ]: print(eulerian_path)
```

```
[ ]: print(len(eulerian_path))
print(eulerian_path[::-1])
```

```
[ ]: ordered_ep = eulerian_path[::-1]

path = []
all_visit = []

for n in ordered_ep:
    if n not in all_visit:
        path.append(n)
        all_visit.append(n)

print(path) #can rename to tsp to consolidate

```

```
[ ]: print(len(path))
# add the starting node to the end of the path to complete the full tsp cycle.
path.append(path[0])

```

2649

time: 616 µs (started: 2025-08-23 03:28:05 +00:00)

```
[ ]: tsp = path
print('IDs:\n' + '-' * 10 + '\n' + '\n'.join(f"{i}: {mst.
↪vs[i]['display_name']}" for i in tsp))

```

IDs:

0: Census Tract 480302

2: Census Tract 480304

...

13: Census Tract 481002

0: Census Tract 480302

time: 4.32 ms (started: 2025-08-23 03:28:08 +00:00)

```
[ ]: hop_distances = [] ##refer to slides
# this is to get the total travel time
for t in range(len(tsp)-1):
    hop = g.distances(source = tsp[t], target = tsp[t+1], weights = 'weight')
    hop_distances.append(hop[0][0])

```

```
print('Hop distances:\n' + '-' * 10 + '\n' +
      '\n'.join(['{: .3f}'.format(x) for x in hop_distances]))
```

Hop distances:

129.765

...

118.335

time: 1min 46s (started: 2025-08-23 03:28:13 +00:00)

4.1.4 Measuring Relative Performance Guarantee

```
[ ]: mst_cost = sum(mst.es['weight'])
```

time: 862 μ s (started: 2025-08-23 03:30:06 +00:00)

```
[ ]: print(mst_cost)
```

269084.545

time: 465 μ s (started: 2025-08-23 03:30:08 +00:00)

WARNING: SLOW!

```
[ ]: tsp_cost = 0
for i in range(len(tsp)):
    if i % 100 == 0:
        print(i)

    #write code
```

```
[ ]: tsp_cost = sum(hop_distances)
print(tsp_cost)
```

422811.085

time: 642 μ s (started: 2025-08-23 03:30:14 +00:00)

4.2 Q12. Find an upper bound on the empirical performance of the approximate algorithm

ρ = Approximate TSP Cost / Optimal TSP Cost

```
[ ]: print('MST Cost:', round(mst_cost))
print('TSP Cost:', round(tsp_cost))
print('Ratio:', round(tsp_cost/mst_cost, 2))
```

MST Cost: 269085

TSP Cost: 422811

Ratio: 1.57

time: 757 μ s (started: 2025-08-23 03:30:23 +00:00)

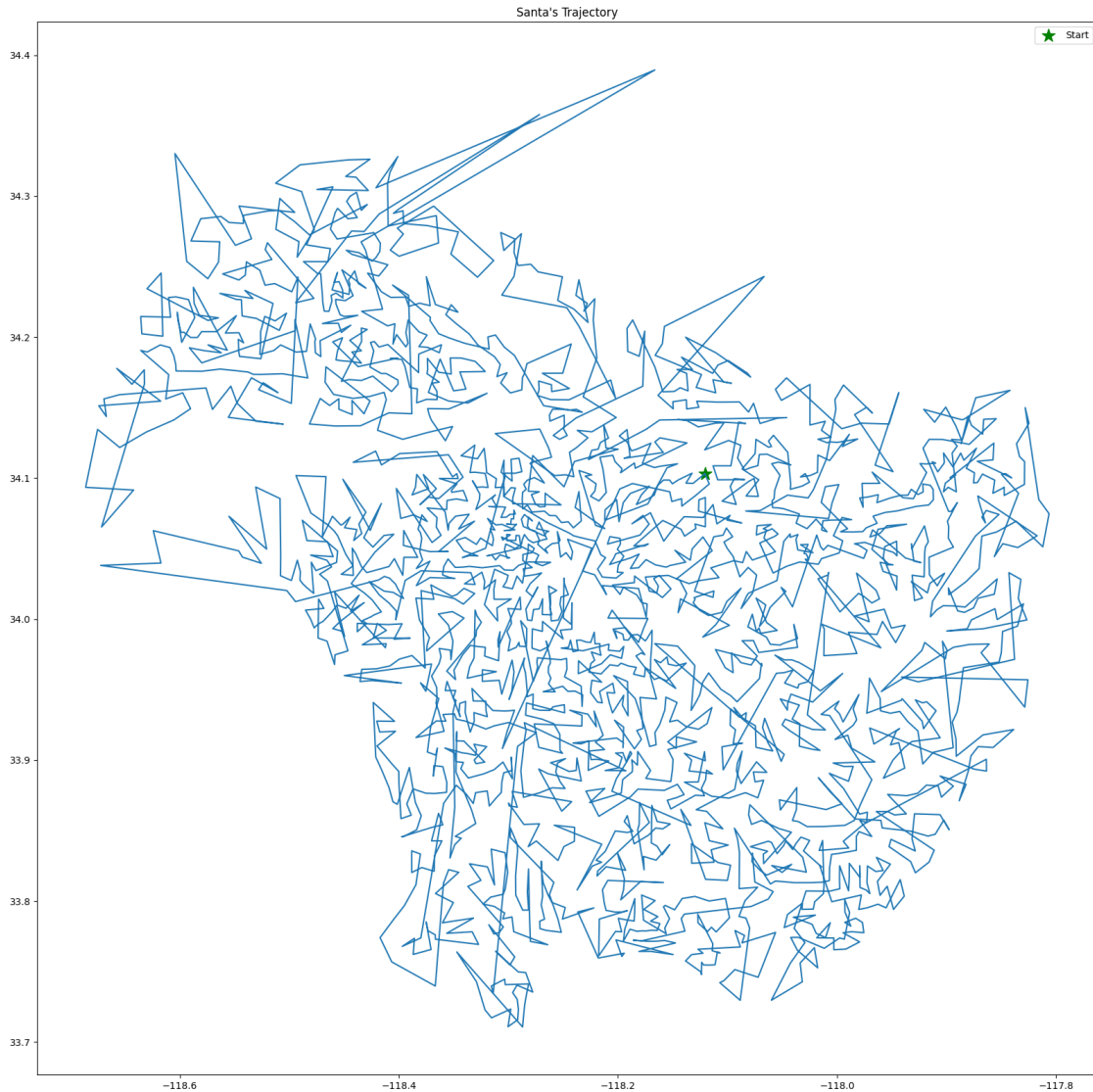
Answer: The upper bound (ρ) should be 2 because the length of tour (tsp_cost) $\leq 2 \times$ Weight of minimum spanning tree (mst_cost) The ratio of 1.57 is in the expected range.

4.3 Q13. Plot the trajectory that Santa has to travel!

```
[ ]: all_coordinates = np.array(g.vs['coordinates'])
     tsp_coordinates = all_coordinates[tsp, :]
     tsp_coordinates = np.vstack([tsp_coordinates, tsp_coordinates[0, :]]) # connect_
     ↪ the last element back to the first element
```

time: 3.21 ms (started: 2025-08-23 03:30:26 +00:00)

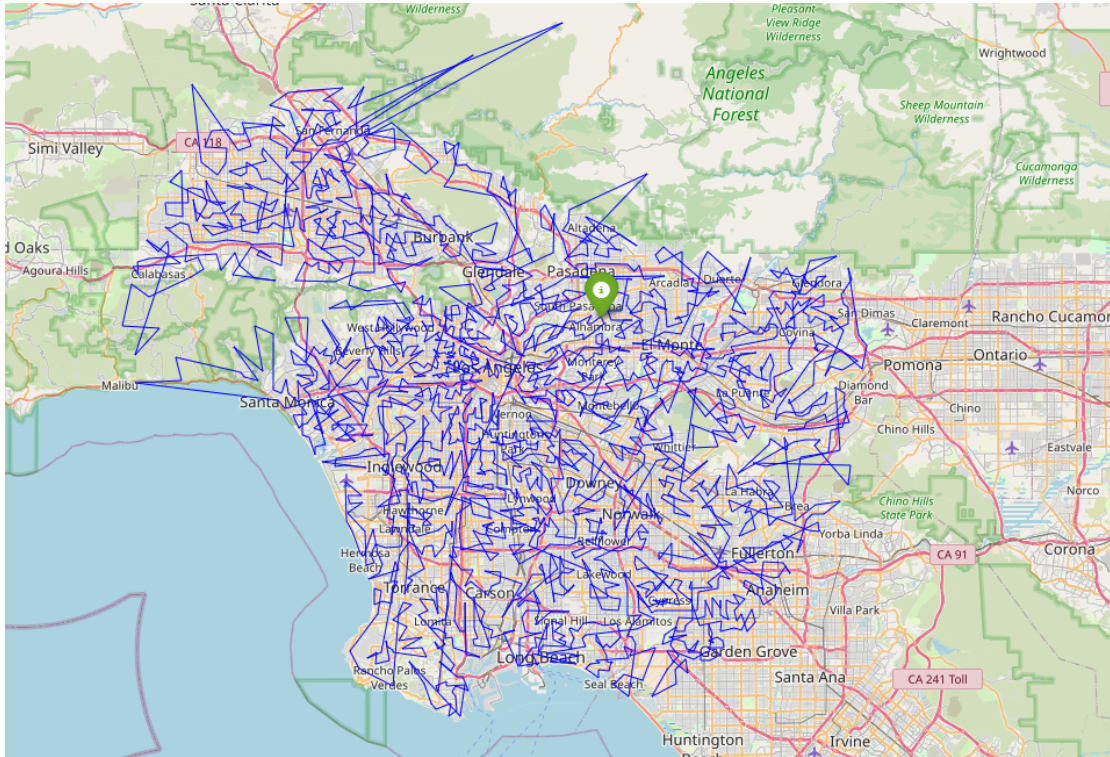
```
[ ]: plt.figure(figsize=(20,20))
     plt.scatter(tsp_coordinates[0, 0], tsp_coordinates[0, 1],
                 color="green", s=200, marker="*", label="Start")
     plt.plot(tsp_coordinates[:, 0], tsp_coordinates[:, 1], '-')
     plt.title("Santa's Trajectory")
     plt.legend()
     plt.show()
```



time: 402 ms (started: 2025-08-23 03:30:28 +00:00)

```
[ ]: # put tour on a map
tourmap = folium.Map(location=(tsp_coordinates[0, 1], tsp_coordinates[0, 0]),
    ↳zoom_start=10, width="60%", height="80%")
folium.Marker(location=[tsp_coordinates[0, 1], tsp_coordinates[0, 0]],
    ↳popup=f"Start", icon=folium.Icon(color="green")).add_to(tourmap)
route_latlon = [(lat, lon) for lon, lat in tsp_coordinates]
folium.PolyLine(route_latlon, color="blue", weight=1).add_to(tourmap)
tourmap
```

```
[ ]:
```



time: 47.6 ms (started: 2025-08-23 03:30:32 +00:00)

5 Analyzing Traffic Flow

Next December, there is going to be a large group of visitors traveling between a location near Malibu to a location near Long Beach. We would like to analyze the maximum traffic that can flow between the two locations.

6 Estimate the Roads

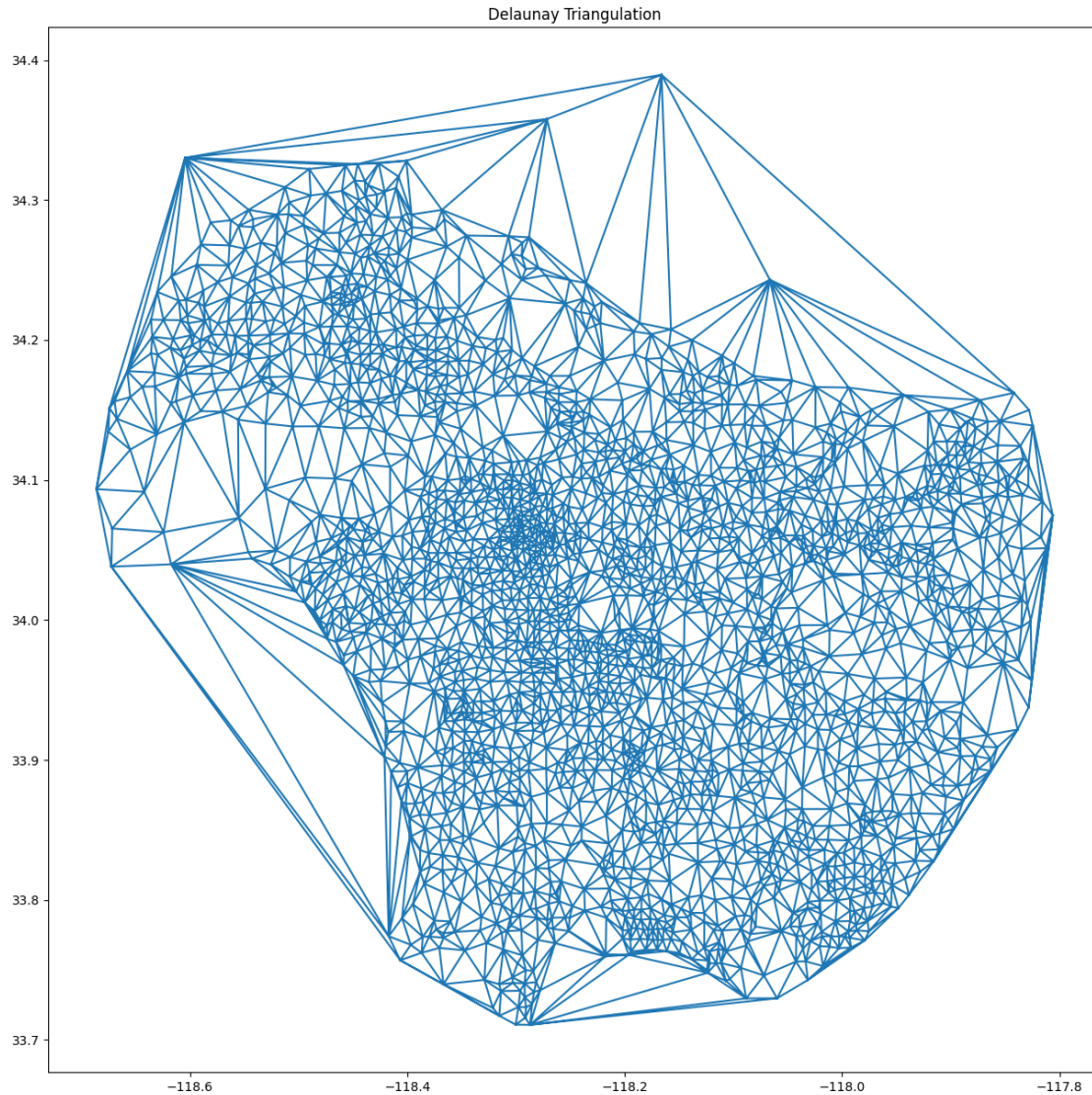
6.1 Q14. Plot the road mesh that you obtain and explain the result. Create a graph $G\Delta$ whose nodes are different locations and its edges are produced by triangulation.

6.1.1 Triangulation

```
[ ]: tri = Delaunay(all_coordinates) #use the Delaunay function imported above
```

time: 13.3 ms (started: 2025-08-23 03:30:36 +00:00)

```
[ ]: plt.figure(figsize=(15,15))
plt.triplot(all_coordinates[:, 0], all_coordinates[:, 1], tri.simplices.copy())
plt.title("Delaunay Triangulation")
plt.show()
```

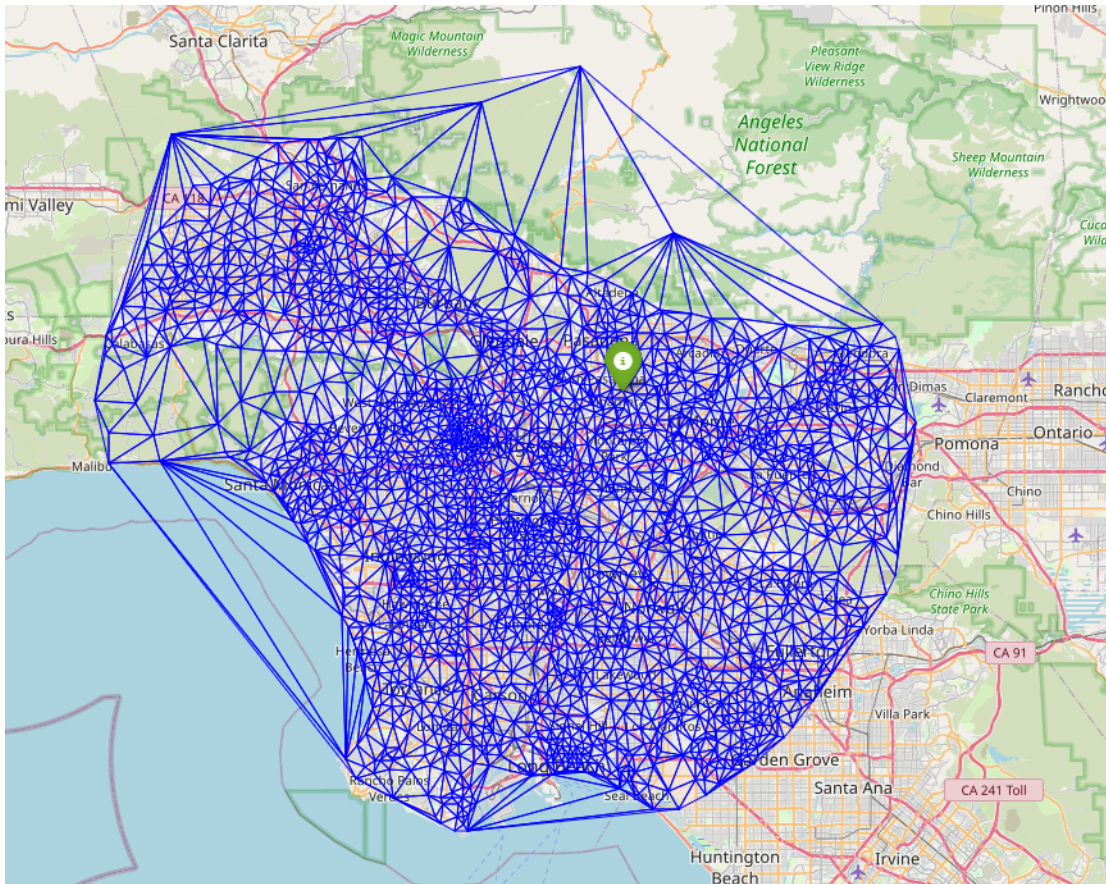


time: 394 ms (started: 2025-08-23 03:30:38 +00:00)

```
[ ]: # put tour on a map
tourmap = folium.Map(location=(all_coordinates[0, 1], all_coordinates[0, 0]),
    ↪zoom_start=10, width="60%", height="80%")
folium.Marker(location=[all_coordinates[0, 1], all_coordinates[0, 0]],
    ↪popup=f"Start", icon=folium.Icon(color="green")).add_to(tourmap)
for coord in tri.simplices:
    pts = [all_coordinates[i] for i in coord]
    edges = [(pts[0], pts[1]), (pts[1], pts[2]), (pts[2], pts[0])]
    for (lon1, lat1), (lon2, lat2) in edges:
        folium.PolyLine([(lat1, lon1), (lat2, lon2)], color="blue", weight=1).
        ↪add_to(tourmap)
tourmap
```



```
[ ]:
```



time: 9.17 s (started: 2025-08-23 03:30:42 +00:00)

Explain the result

Answer: The road mesh plotted is similar to the actual road network in LA. We see that in areas where the nodes are more dense there are also more edges, which mimics real life roads connecting different locations. On the periphery of the road mesh, the nodes become more sparse and the edges span longer distances as well. These edges are like freeways or longer roads that connect distances that are further apart.

```
[ ]: edges_to_induce = []

for i in range(tri.simplices.shape[0]):
    for col1, col2 in ((0, 1), (1, 2), (0, 2)):
        edge = (tri.simplices[i, col1], tri.simplices[i, col2])

        # check if edge exists in g
        if g.are_adjacent(edge[0], edge[1]):
            #add the edges to edges_to_induce
            edges_to_induce.append(edge)
```

time: 25.7 ms (started: 2025-08-23 03:31:04 +00:00)

```
[ ]: print(edges_to_induce)
```

```
[ ]: tri_g = g.subgraph_edges(edges_to_induce)
```

time: 73.5 ms (started: 2025-08-23 03:31:11 +00:00)

7 Calculate Road Traffic Flows

7.1 Q15. Using simple math, calculate the traffic flow for each road in terms of cars/hour. Report your derivation.

7.1.1 Estimate Traffic Flows

```
[ ]: edge_ends = tri_g.get_edgelist()
sources, targets = [coord[0] for coord in edge_ends], [coord[1] for coord in
    edge_ends]

distances = []
for e in tri_g.es:
    distances.append(linalg.norm(tri_g.vs[e.source]['coordinates'] - tri_g.vs[e.
    target]['coordinates'])* 69)

times = np.array(tri_g.es['weight'])/3600 #seconds converted to hours
speeds = distances/times # mph
car_length = 0.003 # (miles)
safety_distances = speeds * (2/3600) # in miles, derived from 2 sec distance
cars_per_mile = distances/(car_length + safety_distances)
n_lanes = 2
cars_per_hour = cars_per_mile * speeds * n_lanes
```

time: 62.9 ms (started: 2025-08-23 03:31:14 +00:00)

```
[ ]: tri_g.es['capacity'] = cars_per_hour
```

time: 814 µs (started: 2025-08-23 03:31:15 +00:00)

```
[ ]: len(cars_per_hour)
```

```
[ ]: 7788
```

time: 2.48 ms (started: 2025-08-23 03:31:17 +00:00)

```
[ ]: print(cars_per_hour)
```

```
[2610.30838923 2320.93506639 1564.10824046 ... 1723.36439509 2680.91204853
1450.17172086]
```

time: 790 µs (started: 2025-08-23 03:31:18 +00:00)

8 Max Flow Analysis

```
[ ]: source_coordinates = [-118.5670548, 34.0458901] # malibu
dest_coordinates = [-118.1819374, 33.759958] # long beach

#malibu = [34.04, -118.56]
#long_beach = [33.77, -118.18]
```

time: 539 μ s (started: 2025-08-23 03:31:20 +00:00)

```
[ ]: # find nearest vertices to source, dest coords (ie. malibu, long beach)
# calculate min distance
def nearest_location(coord):
    min_dist = np.inf

    for i, v in enumerate(tri_g.vs['coordinates']):
        dist = linalg.norm(coord - v)* 69
        if dist < min_dist:
            min_dist = dist
            vertex_idx = i
    return vertex_idx

source = nearest_location(source_coordinates)
dest = nearest_location(dest_coordinates)
print(source)
print(dest)
```

1511

663

time: 27.2 ms (started: 2025-08-23 03:31:23 +00:00)

8.1 Q16. Calculate the maximum number of cars that can commute per hour from Malibu to Long Beach.

8.1.1 Calculate Max Flow

Also calculate the number of edge-disjoint paths between the two spots. Does the number of edge-disjoint paths match what you see on your road map?

```
[ ]: #flow = tri_g.maxflow(source, dest, capacity = tri_g.es['capacity'])

# edge capacity = 1 for edge disjoint paths
print('Number of independent roads:', (tri_g.maxflow(source, dest, capacity = 1)
    ↳ [1]*tri_g.ecount()).value) #max flow of graph (inbuilt is okay )
print('Number of cars per hour:', (tri_g.maxflow(source, dest, capacity = tri_g.
    ↳ es['capacity'])).value)
```

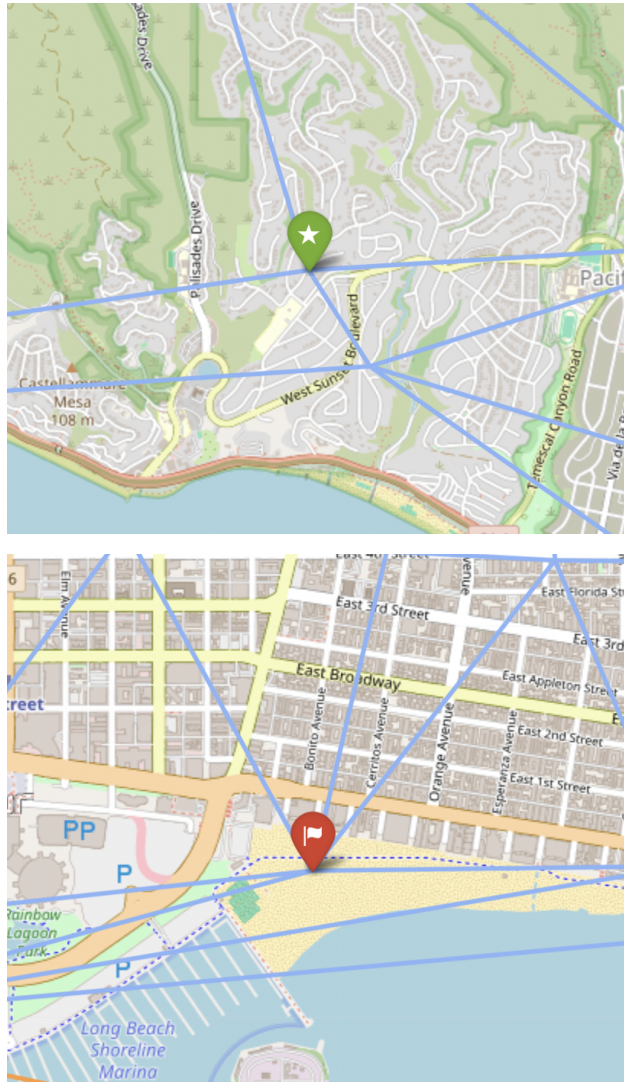
Number of independent roads: 4.0

Number of cars per hour: 12057.035280644373

time: 9.37 ms (started: 2025-08-23 03:31:26 +00:00)

Does the number of edge-disjoint paths match what you see on your road map?

Answer: Yes. We see on the map that Malibu has a degree of 4 edges/roads while Long Beach has a degree of 6 edges/roads. Thus, it is feasible for there to be 4 disjoint paths between Malibu and Long beach.



9 Prune Your Graph

9.0.1 Trimming Large-Distance (Non-Local) Roads

```
[ ]: trim_thresh = 800
```

time: 479 μ s (started: 2025-08-23 03:31:31 +00:00)

```
[ ]: edges_to_induce = tri_g.es.select(weight_le=trim_thresh)
    tri_g_trimmed = tri_g.subgraph_edges(edges_to_induce)
```

time: 8.48 ms (started: 2025-08-23 03:31:32 +00:00)

9.1 Q17. Plot $G\Delta$ on actual coordinates.

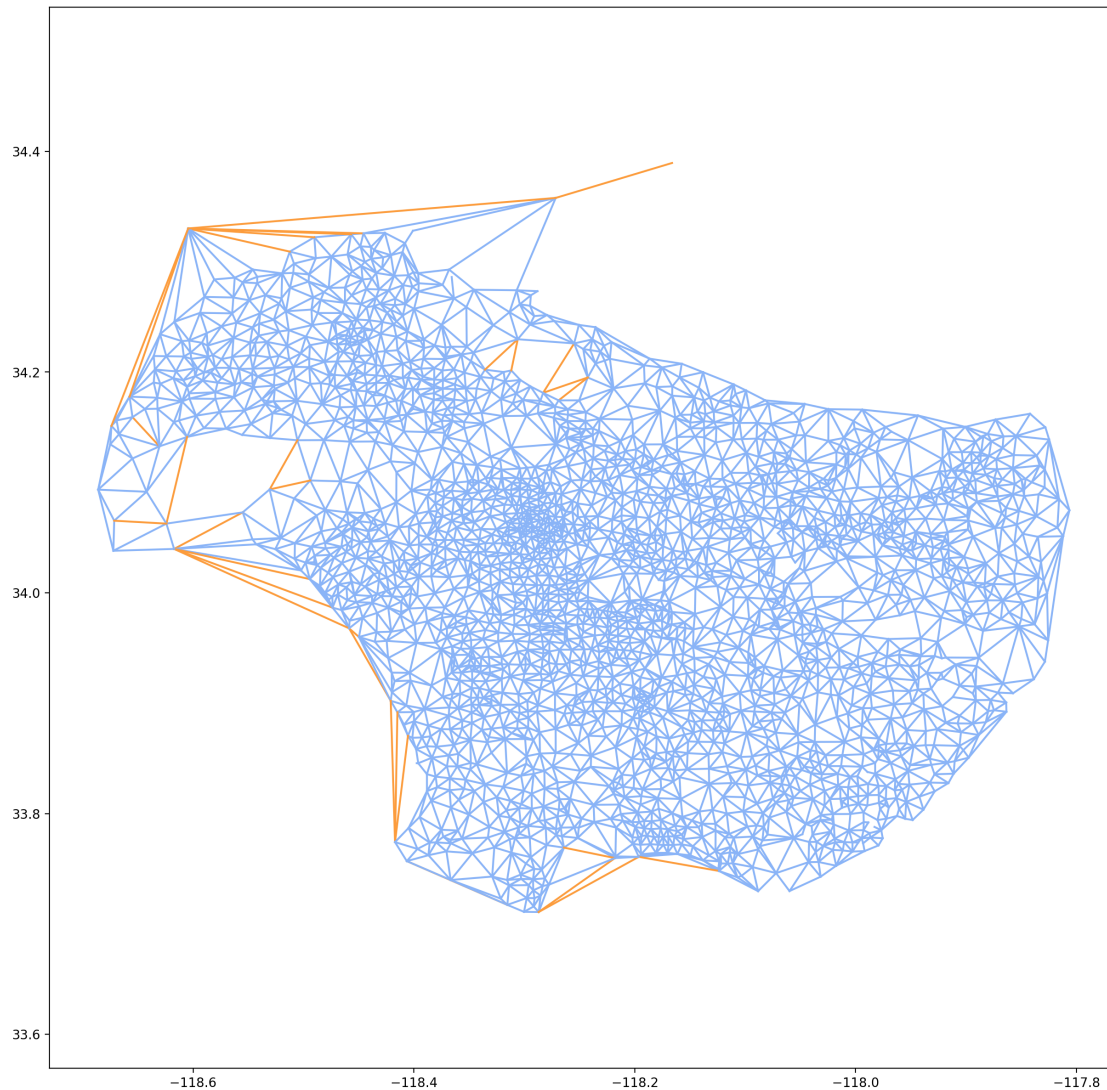
9.1.1 Visualizing Trimming Scheme

WARNING: SLOW!

```
[ ]: # plot the roads
plt.figure(figsize=(15,15), dpi=200)
for e in tri_g.es:
    v1 = tri_g.vs[e.source]['coordinates']
    v2 = tri_g.vs[e.target]['coordinates']
    data = np.vstack([v1, v2])
    x = data[:, 0]
    y = data[:, 1]
    plt.plot(x, y, '#8ab4f7' if e['weight'] < trim_thresh else '#fc9c3c')

# mark the source and destination
# source = tri_g.vs.select(display_name=source_address)[0].index
# target = tri_g.vs.select(display_name=dest_address)[0].index
# v1 = tri_g.vs[source_idx]['coordinates']
# v2 = tri_g.vs[dest_idx]['coordinates']
# data = np.vstack([v1, v2])
# x = data[:, 0]
# y = data[:, 1]
# plt.plot(x, y, 'bo')

plt.axis('equal')
plt.show()
```



time: 5.91 s (started: 2025-08-23 03:31:35 +00:00)

```
[ ]: print('Number of independent roads:', (tri_g_trimmed.maxflow(source, dest,
    ↳ capacity = [1]*tri_g_trimmed.ecount()).value) #max flow of TRIMMED graph
print('Number of cars per hour:', (tri_g_trimmed.maxflow(source, dest, capacity
    ↳ = tri_g_trimmed.es['capacity'])).value)
```

Number of independent roads: 4.0

Number of cars per hour: 12057.035280644373

time: 11 ms (started: 2025-08-23 03:31:51 +00:00)

```
[ ]: map_center = [all_coordinates[:,1].mean(), all_coordinates[:,0].mean()]
roadmap = folium.Map(location=map_center, zoom_start=10, width="60%",
    ↳ height="80%")

# add road edges
for e in tri_g.es:
```

```

v1 = tri_g.vs[e.source]['coordinates'] # (lon, lat)
v2 = tri_g.vs[e.target]['coordinates']
# folium wants (lat, lon)
line = [(v1[1], v1[0]), (v2[1], v2[0])]
color = '#8ab4f7' if e['weight'] < trim_thresh else '#fc9c3c'
folium.PolyLine(line, color=color, weight=1).add_to(roadmap)

# source and destination

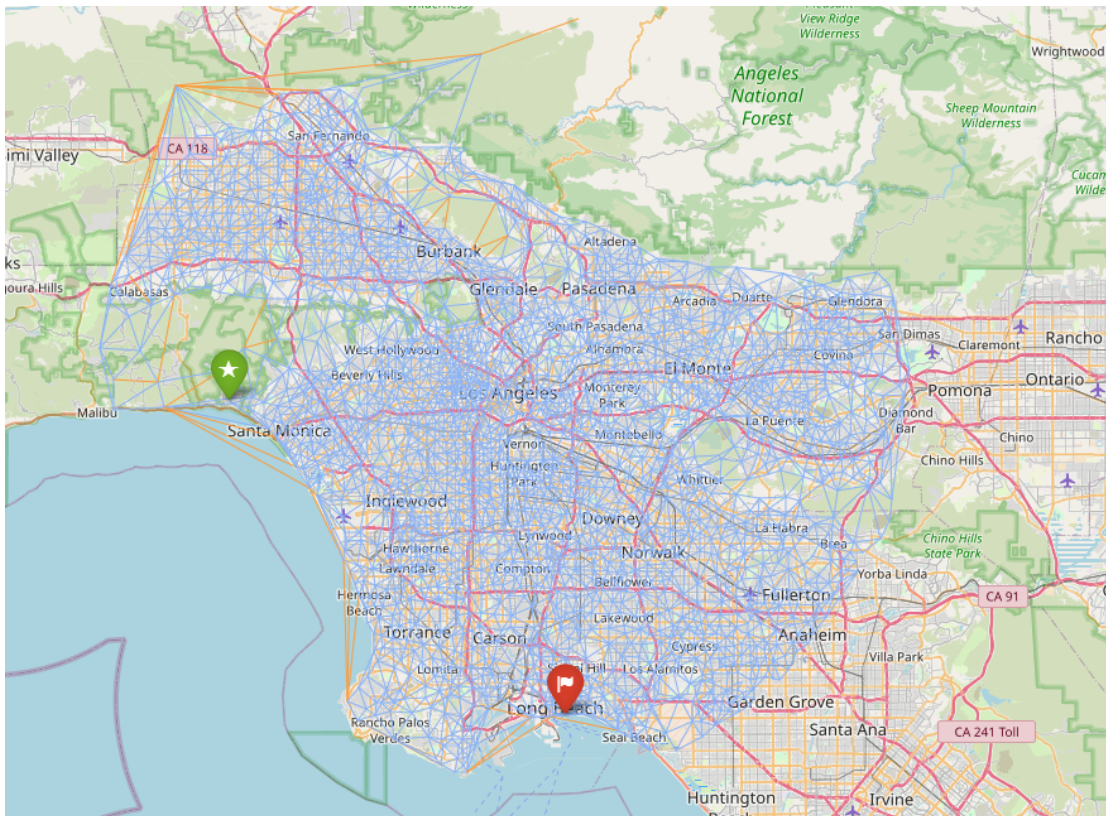
v1 = tri_g.vs[source]['coordinates']
v2 = tri_g.vs[dest]['coordinates']

folium.Marker([v1[1], v1[0]], popup="Source",
              icon=folium.Icon(color="green", icon="star")).add_to(roadmap)
folium.Marker([v2[1], v2[0]], popup="Destination",
              icon=folium.Icon(color="red", icon="flag")).add_to(roadmap)

roadmap

```

[]:



time: 4.36 s (started: 2025-08-23 03:31:53 +00:00)

Do you think the thresholding worked?

Answer: Yes. In the plot, the trimmed edges are in yellow. We see that most of the trimmed edges are on the periphery of the graph. The thresholding seems to have worked - for example some of

the trimmed edges connect locations along the coast, which seem unlikely, seeing as the edges travel across the water.

9.2 Q18. Now, repeat question 13 for $\sim G\Delta$ and report the results.

```
[ ]: #Q10
mst_tri_g = tri_g_trimmed.spanning_tree(weights=tri_g_trimmed.es['weight'])
```

time: 7.4 ms (started: 2025-08-23 03:32:03 +00:00)

```
[ ]: #Q11
n_samples = 1000 # WARNING: this might take a minute for larger sample sizes
triangles2 = 0
sampled_tri = sample_triangles1(tri_g_trimmed, n_samples)

for triangle_vs in sampled_tri:

    edge_lengths = []

    for i in range(3):
        for j in range(i):
            edge_lengths.append(min(tri_g_trimmed.es.
→select(_between=([triangle_vs[i]], [triangle_vs[j]]))['weight']))

    triangles2 += is_triangle(edge_lengths)

print('Triangle inequality holds by {}%'.format(round(100.0 * triangles2 /
→n_samples, 1)))
```

Triangle inequality holds by 82.1%

time: 131 ms (started: 2025-08-23 03:32:05 +00:00)

```
[ ]: eul_tri_g = Graph(directed=False)
eul_tri_g.add_vertices(tri_g_trimmed.vcount())
for edge in mst_tri_g.es:
    source, target = tri_g_trimmed.vs[edge.source].index, tri_g_trimmed.vs[edge.
→target].index
    eul_tri_g.add_edge(source, target)
    eul_tri_g.add_edge(source, target)

stack = [0]
eul_tri_path = []

while stack:
    v = stack[-1]
    if eul_tri_g.degree()[v] > 0:
        for i in eul_tri_g.neighbors(v):
```

```

        #if i not in stack:
            #eid = eulerian_g.get_eid(v, i)
            #eulerian_g.delete_edges(eid)
            #stack.append(i)
            #break

        eid = eul_tri_g.get_eid(v, i)
        eul_tri_g.delete_edges(eid)
        stack.append(i)
        break
    else:
        eul_tri_path.append(stack.pop())

or_tri_ep = eul_tri_path[::-1]

path_tri = []
all_visit2 = []

for n in or_tri_ep:
    if n not in all_visit2:
        path_tri.append(n)
        all_visit2.append(n)

# add the starting node to the end of the path to complete the full tsp cycle.
path_tri.append(path_tri[0])
tsp_tri_g = path_tri
print('IDs:\n' + '-' * 10 + '\n' + '\n'.join(f"{i}: {mst_tri_g.
↳vs[i]['display_name']}" for i in tsp_tri_g))

```

IDs:

```

-----
0: Census Tract 480302
...
0: Census Tract 480302
time: 1.9 s (started: 2025-08-23 03:32:06 +00:00)

```

```

[ ]: hop_dist_tri = []
for t in range(len(tsp_tri_g)-1):
    hop_tri = tri_g.distances(source = tsp_tri_g[t], target = tsp_tri_g[t+1],
↳weights = 'weight')
    hop_dist_tri.append(hop_tri[0][0])

print('Hop distances:\n' + '-' * 10 + '\n' +
      '\n'.join(['{: .3f}'.format(x) for x in hop_dist_tri]))

```

Hop distances:

```

-----
129.765

```

```
...
118.335
```

```
[ ]: mst_cost_tg = sum(mst_tri_g.es['weight'])
      tsp_cost_tg = sum(hop_dist_tri)
```

```
time: 972 µs (started: 2025-08-23 03:32:17 +00:00)
```

```
[ ]: #Q12
      print('MST Cost:', round(mst_cost_tg))
      print('TSP Cost:', round(tsp_cost_tg))
      print('Ratio:', round(tsp_cost_tg/mst_cost_tg, 2))
```

```
MST Cost: 272664
```

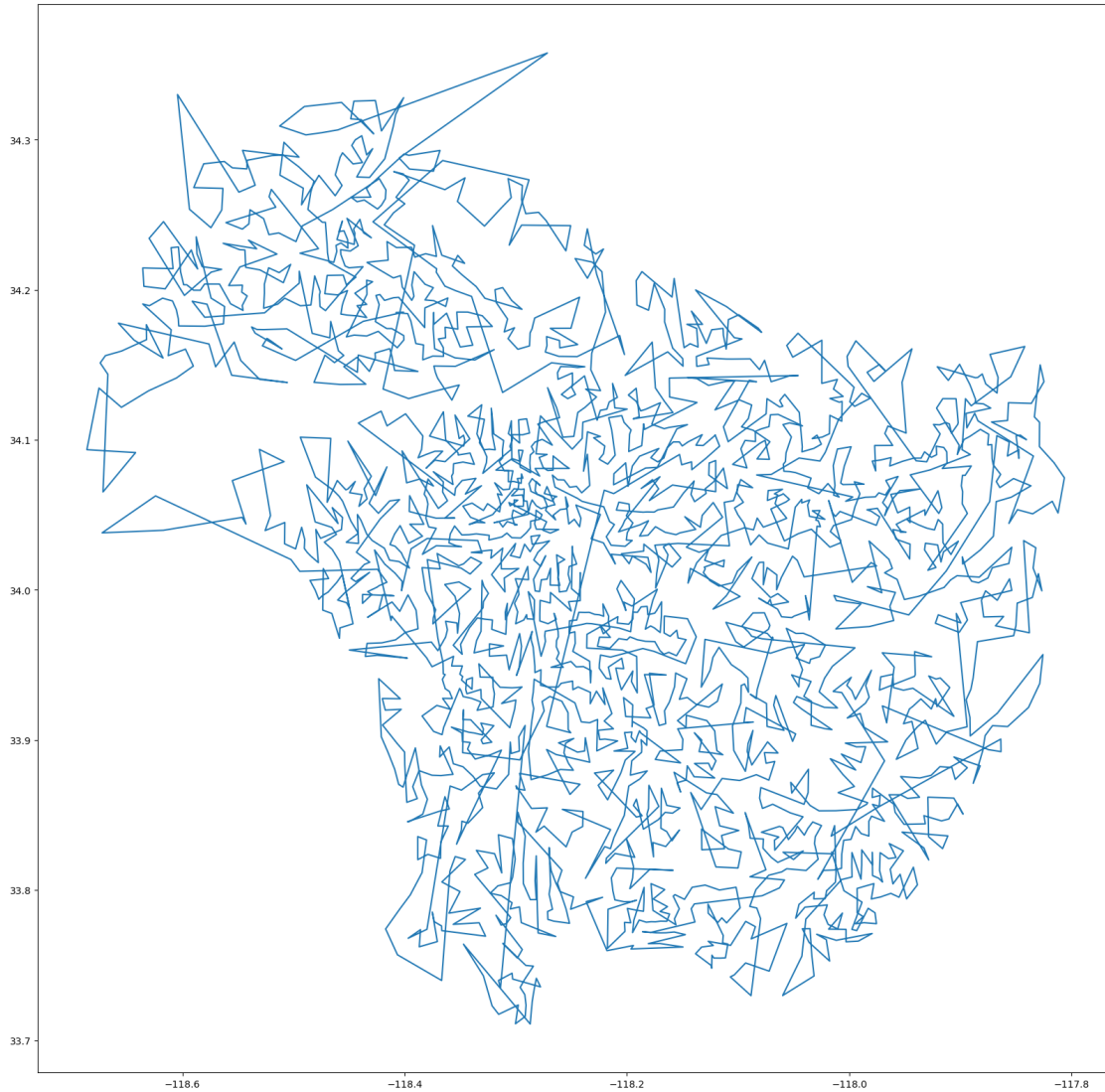
```
TSP Cost: 525863
```

```
Ratio: 1.93
```

```
time: 746 µs (started: 2025-08-23 03:32:19 +00:00)
```

```
[ ]: #13
      all_tg = np.array(tri_g_trimmed.vs['coordinates'])
      tsp_coord_tg = all_tg[tsp_tri_g, :]
      tsp_coord_tg = np.vstack([tsp_coord_tg, tsp_coord_tg[0, :]])

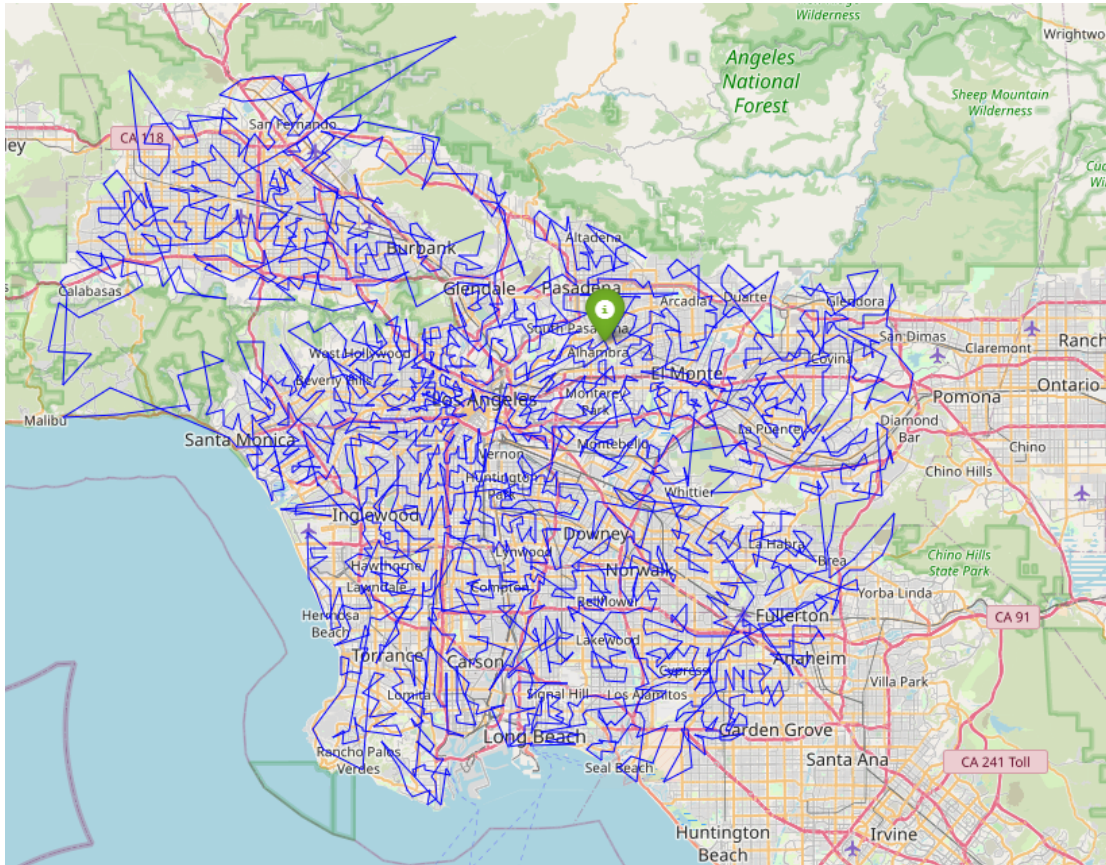
      plt.figure(figsize=(20,20))
      plt.plot(tsp_coord_tg[:, 0], tsp_coord_tg[:, 1], '-')
      plt.show()
```

time: 370 ms (started: 2025-08-23 03:32:20 +00:00)

```
[ ]: tourmap = folium.Map(location=(tsp_coord_tg[0, 1], tsp_coord_tg[0, 0]),
    ↳ zoom_start=10, width="60%", height="80%")
    folium.Marker(location=[tsp_coord_tg[0, 1], tsp_coord_tg[0, 0]],
    ↳ popup=f"Start", icon=folium.Icon(color="green")).add_to(tourmap)
    route_latlon = [(lat, lon) for lon, lat in tsp_coord_tg]
    folium.PolyLine(route_latlon, color="blue", weight=1).add_to(tourmap)
    tourmap
```

[]:



time: 47.3 ms (started: 2025-08-23 03:32:24 +00:00)

Do you see any changes? Why?

Answer: The plot is mostly similar to the original trajectory plotted in Q13, but there are fewer long distances/edges between locations. For example, we can see that there are fewer edges near San Fernando (top left of plot) than in the original trajectory, which makes sense given that we trimmed some edges.

The MST/TSP ratio calculated in this question is higher than the ratio calculated in question 12. This may be due to the fact that we are removing the unrealistic edges in the graph used in question 18. Removing these edges increases the paths between nodes, resulting in longer distances and increased travel times.